Iterative Gaussian–Laplacian Pyramid Network for Hyperspectral Image Classification

Chein-I Chang, *Life Fellow, IEEE*, Chia-Chen Liang, and Peter Fuming Hu

***Abstract*— Gaussian pyramid (GP) is a commonly used image coding technique that encodes an image as a pyramid that is stacked by a set of images with Gaussian window-reduced sizes and multiple spatial resolutions. Associated with GP, a Laplacian pyramid (LP) can be also constructed to represent differential images between images in two consecutive layers of GP. Such resulting Gaussian–Laplacian pyramid (GLP) performs data compression in a lossless and lossy manner. A convolutional neural network (CNN) consists of a series of layers concatenated in a feedforward manner where each layer has a convolutional sublayer (CL) and a pooling sublayer (PL). Interestingly, each layer implemented by CL and PL in a CNN can be realized by a single layer in GP in the sense that CL and PL can be carried out by a low-pass Gaussian filter operated as a Gaussian kernel in a single layer of GP. This article develops a new approach to hyperspectral image classification (HSIC), called the GLP network (GLPN), which uses not only GP to realize CNN but also LP to capture differential information between two consecutive layers that CNN cannot. Furthermore, by incorporating an iterative process into GLPN, we can derive an iterative GLPN (IGLPN) that can be considered a companion of a recently developed iterative random training sampling CNN (IRTS-CNN) by replacing CNN with GLPN. Since GLPN can realize CNN in a better way, it is expected that IGLPN will perform better than IRTS-CNN and also significantly reduce computational efficiency compared to IRTS-CNN.**

***Index Terms*— Convolutional neural network (CNN), deep learning (DL), Gabor filter (GF), Gabor filtered spatial maps (GFSMaps), Gaussian pyramid (GP), Gaussian–Laplacian pyramid (GLP), hyperspectral image classification (HSIC), iterative CNN (ICNN), Laplacian pyramid (LP).**

Manuscript received 5 December 2023; revised 19 January 2024; accepted 13 February 2024. Date of publication 19 February 2024; date of current version 29 February 2024. The work of Chein-I Chang was supported in part by the YuShan Fellow Program sponsored by the Ministry of Education in Taiwan and in part by the National Science and Technology Council (NSTC) under Grant 111-2634-F-006-012. *(Corresponding author: Chia-Chen Liang.)*

Chein-I Chang is with the Center for Hyperspectral Imaging in Remote Sensing (CHIRS), Information and Technology College, Dalian Maritime University, Dalian 116026, China, also with the Remote Sensing Signal and Image Processing Laboratory, Department of Computer Science and Electrical Engineering, University of Maryland, Baltimore, MD 21250 USA, and also with the Department of Electrical Engineering, National Cheng Kung University, Tainan 70101, Taiwan (e-mail: cchang@umbc.edu).

Chia-Chen Liang is with the Remote Sensing Signal and Image Processing Laboratory, Department of Computer Science and Electrical Engineering, University of Maryland, Baltimore, MD 21250 USA (e-mail: lcjane212@gmail.com).

Peter Fuming Hu is with the Department of Anesthesia, R Adams Cowley Shock Trauma Center, Shock Trauma Anesthesia Organized Research Center, University of Maryland School of Medicine, Baltimore, MD 21201 USA (e-mail: phu@som.umaryland.edu).

Digital Object Identifier 10.1109/TGRS.2024.3367127

NOMENCLATURE

3DCNN	3-D convolutional neural network.
CL	Convolutional sublayer.
CNN	Convolutional neural network.
DP	Deep learning.
EPF	Edge-preserving filter.
FCC	Fully connected classification.
GF	Gabor filter.
GFDN	Gabor filtering and deep network.
GFSMaps	Gabor filtered spatial maps.
GP	Gaussian pyramid.
GLP	Gaussian–Laplacian pyramid.
GLPN	Gaussian–Laplacian pyramid network.
GT	Ground truth.
HSIC	Hyperspectral image classification.
IEPF	Iterative edge-preserving filter.
IEPF-G-g	IEPF using a guided filter and the 1st principal component.
IFP	Iterative feedback process.
IGPN	Iterative Gaussian pyramid network.
IGLPN	Iterative Gaussian–Laplacian pyramid network.
ILPN	Iterative Laplacian pyramid network.
IRTS	Iterative random training sampling.
LP	Laplacian pyramid.
PC	Principal component.
PCA	Principal component analysis.
PCAWSD	Pyramid coordinate attention and weighted self-distillation.
PL	Pooling sublayer.
RTS	Random training sampling.
SVM	Support vector machine.
VD	Virtual dimensionality.

I. INTRODUCTION

DEEP learning (DL) has received considerable interest in HSIC due to its DL capability [1], [2]. Specifically, CNN [3] has been developed as one of the most widely used DL techniques, and many works reported in the literature have derived from its framework. As a consequence, these CNN-based approaches are limited to their basic convolutional-pooling structure. To reinvent the wheel, we look into the key idea of CNN, which is the CL and PL implemented in a single layer.

In communications and signal processing, low-pass filtering and subsampling have been widely used to reduce spatial

1558-0644 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

resolution and image size, respectively. Interestingly, if we interpret CL of CNN as a reduction of spatial resolution by a Gaussian low-pass filter and PL of CNN as a reduction of image size by subsampling, then the GP-based image encoding technique developed by Burt and Adelson [4] can realize what the (CL, PL) layer does for CNN. Its idea constructs a pyramid with the bottom layer corresponding to the original image and then implements a 5×5 Gaussian window as a low-pass filter to reduce image spatial resolution and image size simultaneously from one layer to its immediate right above layer in a bottom-up manner. In the meantime, an LP can be also constructed simultaneously by taking the difference of images in two consecutive layers in GP to yield a differential image that forms a layer of LP. An advantage resulting from the constructions of GP and LP over CNN is that LP represents new information that GP does not possess in such a manner that layers of LP can be used to reconstruct the original image. Consequently, the encoding only needs LP for data processing without storing GP. This process is called the GLP method and has been used for lossless or lossy image compression.

A. Motivations

Seemingly, CNN and GLP are completely different paradigms in pattern recognition and communications/signal processing, respectively. A CNN consists of a series of layers concatenated one after another in a feedforward manner where each layer has two sublayers, CL and PL, with the last layer specified by an FCC layer. The key idea of CNN is to utilize CL to extract spatial information from a receptive field centered at each data sample followed by PL to subsample the CL-processed data samples and then the FCC layer to train CNN via adapting weights connecting between layers through backpropagation.

It is interesting to note that the CL and PL used by CNN can be actually realized by GLP as a network, called GLPN. In other words, GP is constructed bottom-up by Gaussian windows that correspond to a series of CL using Gaussian kernels in CNN. On the other hand, each layer of GP is downsampled from its preceding layer and can be considered subsampling used by the PL sublayer in CNN. Finally, the number of layers in GP is indeed the depth of CNN. With this interpretation, it seems that a CNN can be implemented similarly to a GP on one shot-operation without using (CL, PL). However, there are also differences between CNN and GP that make this realization challenging. First, there is an FCC implemented in the last layer of CNN, which is absent in GP. Second, a CNN is determined by a set of adaptive weights that are trained by preselected training samples as opposed to GP, which is constructed by preselected Gaussian windows without training samples. Third, a CNN is primarily used for classification that needs to be trained, whereas a GP is mainly designed for image coding by prefixed Gaussian low-pass filters. Most importantly, a GP can generate an LP that can be used to reconstruct the original image, so as to achieve lossless or lossy image compression. There is no such LP counterpart in CNN. Thus, the motivation of this article is to resolve the abovementioned issues and further develop GLPN in the sense that what a CNN can do with CL and PL can also be accomplished by GP.

B. Proposed Approach

By comparing GLP to CNN, we immediately realize that the CL and PL implemented in a single layer of CNN are essentially the same as a single layer of GP. This suggests that GLPN may be able to do what CNN does, provided that GLPN can be developed appropriately. In doing so, a recent approach, IRTS-CNN developed in [5], may offer a solution. It extends CNN by including a feedback system to repeatedly process the same CNN in an iterative manner so that CNN can be improved by updating its spatial information of classification maps through a feedback spatial filtering system via IRTS. Unfortunately, a direct replacement of GLP for CNN used in IRTS-CNN is not feasible because the input to GLPN must be a single image, not a data cube such as a hyperspectral image (HSI). To resolve this issue, the EPF developed in [6] is applied to the current data cube to produce an EPF map for each particular class. However, it is often the case that such EPF-based classification may incur misclassification. To further reduce the misclassification rate, a set of GFs is further implemented on the EPF map to capture spatial information of EPF-classification maps (i.e., EPF-classified data samples) to produce GFSMaps. These GFSMaps are then used as input images to GLPN to construct GP and LP, which are fed back and added to the current data cube to create a new augmented data cube for the next round iteration. The entire process that is similar to that implemented in IRTS-CNN is repeated iteratively until it satisfies a stopping rule. The structure of IRTS-CNN provides a base to derive an IGLPN, which can be considered a companion of IRTS-CNN.

C. Novelties

The key novelty of this article is the introduction of an IFP into GLPN to derive IGLPN. This process utilizes a set of spatial GFs with eight orientations to capture spatial information from classification maps and then feeds GFSMaps back to expand the current being processed data cube to a new data cube for the next round iteration. This is completely opposite to the feedforward process implemented in CNN. A second novelty is IGLPN that offers a comparative alternative to IRTS-CNN. A third novelty is the construction of LP coupled with GP to provide innovational information about classification, a new concept similar to Kalman filtering developed for hyperspectral imaging [7] that has never been explored in the past. More specifically, the spatial information of classification obtained from GFs can be described by two types of information: classification information predicted by GP and innovational unpredictable information generated through LP. By virtue of Kalman filtering, these two pieces of information can be used to derive a hybrid of GP and LP, called GLP that can further improve classification accuracy.

D. Contributions

The main contributions of this article can be summarized in the following.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

1) This article explores the connection of CNN to GLP and interprets the (CL, PL) of a single layer in CNN as a single layer of GP, which reduces image resolution corresponding to what CL does and downsizes the image corresponding to PL, which subsamples the CL-processed data samples.
2) GLPN constructs GP from the bottom-up and LP from the top-down to capture different levels of spatial information provided by GP. This cannot be done by CNN. In particular, no corresponding LP counterpart can be found in CNN. In other words, the LP constructed by GLP retains the innovational spatial information between images in two consecutive layers in GP. This concept is not used in CNN. It is worth noting that LP is an important part of GLP for image coding, which is the key to reconstructing the original image without restoring GP. It is different from CNN that is mainly designed for pattern classification, not for data reconstruction. This may explain why there is no counterpart of LP in CNN. For CNN to be able to do what LP does, it requires an additional sublayer, called differential sublayer to find a differential image in a similar manner that LP is calculated by GLP. Unfortunately, this will add another computational burn by training this differential sublayer.
3) GLPN is further extended to IGLPN that includes feedback loops of GFSMaps to capture spatial correlation among classified data samples in an iterative process, so as to improve the classification rate.
4) Compared to CNN that is a feedforward network, IGLPN is a feedback network. Thus, an SVM classifier is implemented in IGLPN to replace the FCC layer used in CNN as a stopping rule, which calculates the SVM-classification accuracies between two consecutive iterations. If the difference is within a given error tolerance, the iterative process is then terminated. Furthermore, this SVM classifier can also serve as a decision-maker to determine the number of iterations executed by IGLPN that corresponds to the depth in CNN.

The remainder of this article is organized as follows. Section II provides a brief review of related works. Section III describes two networks: CNN and GLP for comparison. Section IV details the design rationale of IGLPN. Section V develops a detailed algorithm for implementing IGLPN. Section VI describes images to be used for experiments. Section VII presents experiments and discussions. Section VIII includes an ablation study on the efficiency of subcomponents in IGLPN. Section IX draws some conclusions.

II. Related Works

Over the past decade, numerous CNN techniques have demonstrated remarkable performance in the field of HSI classification, such as a hybrid spectral CNN (HybridSN) [8], a spectral–spatial-dependent global learning (SSDGL) [9], an attention-based adaptive spectral–spatial kernel improved residual network (A2S2K-ResNet) [10], and other CNN-HSIC methods [10], [11], [12], [13], [14], [15], [16], [17]. Traditionally, CNN consists of three components, CL, PL, and the FCC layer. The CL is used for feature extraction, while PL is used to reduce data dimensionality and retain desirable information. However, limited training samples and the limited scale diversity of feature extraction are often obstacles for CNN to achieve higher accuracy.

Recently, a new CNN has been developed in [5] to address the challenge of limited training samples that can be used for CNN. It is referred to as the IRTS-CNN that is a framework consisting of three key components: RTS, EPF, and IFP. Such IRTS-CNN can be readily implemented with any CNN classifier. It begins by randomly selecting training samples and employing a CNN classifier. Subsequently, a series of spatial filters are applied to capture the spatial information of the classified data samples. This spatial filter-captured information is then iteratively fed back into the process to expand the current input data cube as a new data cube that will be used for the next round iteration to randomly reselect training samples and reimplement the CNN. This entire process is repeated over and over again until a predefined stopping rule is satisfied. However, while IRTS-CNN demonstrates impressive performance, it does come with the drawback of increased computation time.

To address the high computational time associated with the standard CNN-HSIC, Gabor filtering-based CNN methods have been recently developed. Kang et al. [18] introduced the GFDN that combines the GF with the stacked sparse autoencoder (SSAE) deep network. In addition, they developed a virtual sample constructing method to address the issue of limited training samples. In another study, Liu et al. [19] proposed a novel Gabor network that employs Gabor filtering for convolutions in CNNs, thereby reducing the number of parameters to learn. Zhao et al. [20] presented the fractional Gabor convolutional network (FGCN), which introduces the fractional Gabor convolutional (FGC) layers designed to extract multiscale, multidirectional, and semantically varying features. Experimental results from their study suggest that incorporating Gabor features can enhance the performance of CNNs.

In addition, many multiscale feature extraction techniques have been developed including wavelet transform and pyramid coding to solve the aforementioned problems. For example, Ghasrodashti et al. [21] proposed a method that included a sparse classifier called optimum dictionary learning (ODL), applied in the nonsubsampled pyramid domain to take advantage of the multiresolution property of pyramids, and an extended hidden Markov random field (EHMRF). Paoletti et al. [22] applied pyramidal bottleneck residual blocks in the residual-based network. This approach not only involves more locations as the network depth increases, thus balancing the workload among all units, but also maintains the time complexity per layer. Tu et al. [23] developed a multiresolution pyramid-enhanced nonlocal feature extraction (MRPEN) method by using nonlocal information from different resolutions to reduce the effects of unwanted information. He et al. [24] derived an object-oriented classification method that was based on the cooperation between superpixel segmentation and 3-D spatial pyramid pooling (3-D-SPP).

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

Similarly, pyramidal convolution (PyConv) [25] employs a pyramid of diverse convolutional kernels, producing a series of output multiscale feature maps. Yue et al. [26] adopted adaptive spatial pyramid constraint (ASPC) to take advantage of the global spatial neighborhood information of the labeled samples. Shi et al. [27] proposed an improved dense block based on a multiscale spectral pyramid (MSSP) using the concept of multiscale and group convolution of the convolution kernel. This approach can fully extract spectral information from HSIs. Shang et al. [28] combined the pyramidal coordinate attention and weighted self-distillation (PCAWSD) to create a novel HSI classification algorithm. Furthermore, Song et al. [29] proposed a novel lightweight HSIC method using a successive spatial rectified network (SSRNet), which consists of the spectral full convolutional network and several separable spatial pyramid modules, along with a noncentral positional sampling (NCPS) strategy. These methods used various pyramid-related methods to extract the multiscale features, which are different from the image pyramid that we used in the proposed method. Tang et al. [30] proposed a dual-branch feature extraction network consisting of GP multiscale transformation and wavelet transform, which is adopted for multiscale feature extraction and reducing the influence of abnormal spectral data. Li et al. [31] also utilized GP coupled with segmented PCA to develop a multiscale feature fusion method for HSIC. Instead of using a Gaussian filter, Hao et al. [32] developed a curvature filter-based multiscale feature extraction method with multiscale superpixel segmentation constraint. This method uses the idea of an image pyramid decomposition method for the global multiscale curvature features (GMCFs) and fuses it with the superpixel segmentation-based local multiscale spatial features (LMSFs) for classification purposes. Although these multiscale CNN methods show promising results, they are still limited to their basic convolutional-pooling structure, which comes with several inherent challenges. One challenge is the difficulty in finding the optimal parameter settings for DL techniques, which can significantly impact performance. In addition, these existing methods often demand considerable computational time and high computer environments, which may limit their practical applicability. In particular, none of these methods took advantage of LP. To address these issues, IGLPN is designed to not only emulate the structure of CNN but also improve its efficacy and enhance classification accuracy while reducing computational time.

III. Review of CNN and GLP

This section reviews two networks that look dissimilar but are indeed closely related in terms of their design rationales. Since these two networks have been studied extensively in their own areas, only their basic network structures are discussed and their close connections are explored. For those who are interested in these networks, we refer details to the current existing literature.

A. CNN

CNN has emerged as a major technique in DL networks and is well-studied in the literature, and numerous references are available in the public domain. Its basic architecture is shown in Fig. 1 [33], which consists of a series of hidden layers with CL and PL implemented in each layer and concluded by an FCC layer as its output layer.

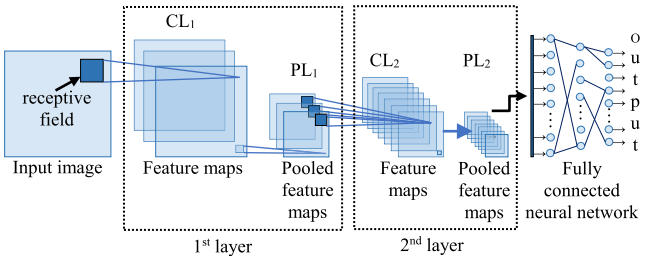


Fig. 1. Basic stricture of CNN.

It is analogous to that of the connectivity pattern of neurons in the human brain and is inspired by the organization of the visual cortex where only two layers are illustrated with a fully connected neural network. Individual neurons respond to stimuli only in a restricted region of the visual field known as the receptive field. For a detailed tutorial reference, we refer to [33, Sec. 13.6].

B. Gaussian–Laplacian Pyramid

GLP was first developed by Burt and Adelson [4] for image encoding. It constructs GP for a given image $\mathbf{G}_0$ shown in Fig. 2(a) where the image at the bottom layer is $\mathbf{G}_0$ and for $l \geq 0$ the image at the $(l+1)$th layer, $\mathbf{G}_{l+1}$ is a size-reduced image of the image at the lth layer, $\mathbf{G}_l$ via a reduce operator

$$\mathbf{G}_{l+1} = \text{RED}(\mathbf{G}_l). \tag{1}$$

Associated with GP, LP can be constructed by taking the difference between the images at two consecutive images in GP in Fig. 2(a). Since $\mathbf{G}_{l+1}$ and $\mathbf{G}_l$ have different sizes, $\mathbf{G}_{l+1}$ needs to be expanded as an image, denoted by $\tilde{\mathbf{G}}_l$, with the same size as $\mathbf{G}_l$. For example, to construct the lth layer, $\mathbf{L}_l$, of LP for $l \geq 0$, an operator, EXP, is needed to expand $\mathbf{G}_{l+1}$ to $\tilde{\mathbf{G}}_l$ via

$$\tilde{\mathbf{G}}_l = \text{EXP}(\mathbf{G}_{l+1}) \tag{2}$$

which matches the same size of $\mathbf{G}_l$. Then, a differential image can be obtained by finding $\mathbf{L}_l = \mathbf{G}_l - \tilde{\mathbf{G}}_l$ shown in Fig. 2(b).

1) GP Construction: The bottom base of GP is the original image, represented as $\mathbf{G}_0$. For a level $l \geq 0$, the value of the pixel in the $(l+1)$th image, $\mathbf{G}_{l+1}$ at the spatial location (i, j) is computed as a weighted average of values of the pixels within a 5×5 window in the lth image, and $\mathbf{G}_l$ at the spatial location (i, j) is computed as follows. The level-to-level averaging process is computed by the function RED as follows.

$$\mathbf{G}_{l+1}(i, j) = \sum_{m=-2, n=-2}^{2,2} w(m, n)\mathbf{G}_l(2i + m, 2j + n) \tag{3}$$

where $w(m, n) = w(m)w(n)$, $w(m) = w(-m)$ for $m = 0, 1, 2$, and $\sum_{m=-2}^{2} w(m) = 1$. In this case, (3) can be defined as

$$\mathbf{G}_{l+1}(i, j) = \text{RED}(\mathbf{G}_l(i, j)). \tag{4}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

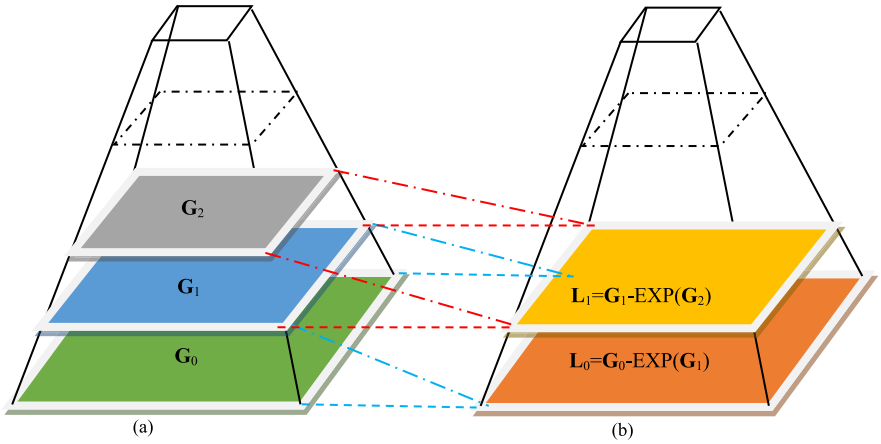


Fig. 2. GLP construction. (a) GP. (b) LP.

Specifically, according to [4], $w(m)$ was suggested to be

$$w(0) = a, \quad w(1) = w(-1) = 1/4$$
$$w(2) = w(-2) = 1/4 - a/2 \tag{5}$$

where a is a parameter to control the shape of the equivalent weighting function. According to Burt and Adelson [4], $a = 0.4$ was set for experiments.

2) LP Construction: Since the images in two different layers in GP have different image sizes and LP is constructed by taking the difference between the images in two different layers in GP, an issue arising how to resize images in two different layers at the same size, in this case, the EXP operator in (2) is defined in [4] as

$$\tilde{\mathbf{G}}_l(i, j) = \text{EXP}(\mathbf{G}_{l+1}(i, j)) \tag{6}$$

where

$$\tilde{\mathbf{G}}_l(i, j) = 4 \sum_{m=-2, n=-2}^{2,2} w(m, n) \mathbf{G}_{l+1}\left(\frac{i-m}{2}, \frac{j-n}{2}\right). \tag{7}$$

It should be noted that (6) can be considered the reverse of (4).

Once the images in two different layers are resized by (6), for each $l \geq 0$, we can constrict the lth image of LP at level l by

$$\mathbf{L}_l = \mathbf{G}_l - \text{EXP}(\mathbf{G}_{l+1}) = \mathbf{G}_l - \tilde{\mathbf{G}}_l. \tag{8}$$

C. Interpretation of GP as CNN

Now, if we pay attention to the 5×5 window used in (3), it works like a Gaussian kernel. As a consequence, the RED operator using a Gaussian kernel in (3) plays the same role as CL does for CNN in Fig. 1. On the other hand, the RED operator that reduces the image $\mathbf{G}_l$ to the image $\mathbf{G}_{l+1}$ can be also considered subsampling operator carried out by PL for CNN in Fig. 1. Accordingly, the number of layers in GP corresponds to the depth of CNN. With this interpretation, GP can be used to realize CNN. Interestingly, there is no corresponding counterpart LP to be found in CNN.

IV. Design of IGLPN

As noted in Section III, GP processes an image by gradually reducing image sizes via Gaussian windows as low-pass filters to construct a pyramid in a progressive fashion, and in the meantime, it also reduces image spatial resolution layer by layer. This process is similar to that used in CNN that makes use of CL to capture spatial patterns via kernels operating on receptive fields and PL via subsampling to reduce image size. Despite the fact that the kernels used in CL are not necessarily Gaussian kernels like that used in GP, their similarity inspires a development of GLPN that can be particularly designed to better describe the human visual system than CNN does.

A. Idea of IGPLN

Technically speaking, designing IGLPN is feasible. However, there are two issues that need to be resolved. One is how to realize CL and PL operated in a single layer of CNN by Gaussian windows and downsample images used in GLPN. The other is that there is no corresponding FCC layer used in CNN that can be realized by GP since GP was originally developed for image encoding not classification.

Fortunately, a recently developed IRTS-CNN [5] provides a feasible solution. However, a direct use of IRTS-CNN is not applicable. We need to reinvent the wheel to explore how a GP can match the infrastructure of CNN. In doing so, we take advantage of the structure of IRTS-CNN as a basic network while replacing CNN with GP. However, this seems easier said than done because such a straightforward replacement does not work. One key issue is how to take care of the training sample-trained FCC layer used in CNN, which is not applicable to GP. This is because the FCC used in CNN is trained by training samples, whereas IGLPN is a feedback network without training. To resolve this issue, the FCC is replaced by EPF-based classifiers developed in [6], which preserve boundaries among data samples so that data samples nearby boundaries can be better classified. Then, the EPF classification maps can be used to replace FCC for GP. In order to emulate the human visual system, a GF with eight different orientations is applied to the EPF maps to produce eight GFSMaps to capture spatial orientations of classified samples

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

for each EPF classification map. These GFSMaps serve two purposes. One is used for the feedback process to be added to the current data cube to create a new augmented data cube. The other is used to determine whether or not the iterative process should be terminated. The whole EPF-GFSMaps process is repeated over again iteratively until it satisfies a stopping rule. The network described above is called IGLPN.

1) EPF-Based Classifier: EPF-based approaches developed in [6] have been widely used in the field of HSIC in recent years. For instance, Zhong et al. [34] developed an IEPF to improve classification accuracy. Duan et al. [35] proposed an edge-preserving operation (EPO) to fuse the edge-preserving feature produced by local and global EPF. Kang et al. [36] developed a PCA-based multiscale edge-preserving filtering (PCA-EPF) method for feature extraction of HSI, in which the multiscale edge-preserving features are used for classification.

In general, there are four EPF classifiers, EPF-G-g, EPF-G-c, EPF-B-g, and EPF-B-c proposed in [6], among which EPF-G-g seemed to perform well compared to the other three. In this case, EPF-G-g was selected for experiments. It used a guided filter with the first PC image of the HSI data cube as a reference image. The guided filter [37] was originally conceived from a local linear model that seeks to smoothen images while maintaining their edge structure by considering a guidance image. This guidance image can either be the input image itself or a different image.

Initially, EPF-G-g first employs SVM to generate a set of initial spectral-based SVM classification maps, $\{B_{\mathrm{SVM},j}\}_{j=1}^{M}$. Subsequently, a guided filter is applied to SVM maps to yield an EPF map for each specific class, $\{\mathrm{EPF}_j\}_{j=1}^{M}$, which captures spatial information from the SVM-classified data samples. Since the EPF map is a soft probability map, maximum a posteriori (MAP) is then used to generate binary classification maps from $\{\mathrm{EPF}_j\}_{j=1}^{M}$, $\{B_{\mathrm{EPF},j}\}_{j=1}^{M}$, which will be used to calculate a stopping criterion to determine if IGLPN needs to be terminated.

2) Gabor Filter: The GF is a 2-D Gaussian distribution and has been shown in [38] to be able to mimic the human visual system. Since the (GP, LP) layers in GLP are designed to emulate the human visual system for this purpose, GF is used as the desired spatial filter for IGLPN to capture spatial information of eight orientations from EPF maps.

GF has been widely used in image processing and computer vision applications. It draws inspiration from the receptive fields of simple cells found in the primary visual cortex of the human brain. These cells selectively respond to specific visual patterns, such as edges and textures, at particular orientations and spatial frequencies. In the spatial domain, the GF can be defined as follows [39]:

$$h(x,y)=\exp\left\{-\frac{1}{2}\left[\frac{x^2}{\sigma_x^2}+\frac{y^2}{\sigma_y^2}\right]\right\}\cos(2\pi u_0 x+\phi) \tag{9}$$

where u_0 and ϕ are the frequency and phase of the sinusoidal plane wave along the x-axis and σ_x and σ_y are space constants of the Gaussian envelope along the x-axis and the y-axis, respectively. For the GF in the frequency domain, its Fourier transform formula when $\phi=0$ can be expressed as [40]

$$H(u,v)=2\pi\sigma_x\sigma_y\left(\exp\left\{-\frac{1}{2}\left[\frac{(u-u_0)^2}{\sigma_u^2}+\frac{v^2}{\sigma_v^2}\right]\right\}+\exp\left\{-\frac{1}{2}\left[\frac{(u+u_0)^2}{\sigma_u^2}+\frac{v^2}{\sigma_v^2}\right]\right\}\right) \tag{10}$$

where $\sigma_u=(1/2\pi\sigma_x)$, $\sigma_v=(1/2\pi\sigma_y)$, $\sigma_x=(\lambda/\pi)(\ln 2/2)^{1/2}\times(2^b+1)/(2^b-1)$, $\sigma_y=\sigma_x/\gamma$, b is the bandwidth, γ is the spatial aspect ratio, λ is the wavelength, (x, y) are the spatial coordinates representing pixel positions, and (u, v) is the frequency coordinates representing the features of the GF in the frequency domain.

B. Uniqueness of IGLPN

There are several unique features that distinguish IGLPN from CNN and can be summarized in the following.

1) *Network Structure:* The first and foremost is that CNN can be realized by GLPN with a difference in the sense that CNN is carried out by a cascade of size-reduced networks, whereas GLPN is executed by a bottom-up network with each layer in CNN indeed corresponding to one layer of GP.
2) *Feature Extraction Layer:* GP can be also used to construct LP where each layer of LP is an innovational layer, which represents a differential layer between two consecutive layers in GP. In contrast, CNN does not have such layers corresponding to layers in LP.
3) *Feedforward Versus Feedback:* CNN is a feedforward network as opposed to IGLPN, which is an iterative feedback network. In this case, CNN can be considered one iteration of IGLPN.
4) *Spatial Information:* Since the input to each layer of CNN is directly obtained by PL in its preceding layer, there are no spatial filtered maps included in the input layer compared to IGLPN, which generates GFMaps to capture spatial information among classified samples and then feed these GFMaps back and added to the current data cube to create a new augmented data cube for next round iteration
5) *Classification:* CNN only performs classification in its last FCC layer. The purpose of this layer is to use training samples to train CNN via backpropagation to adjust weights connected between layers. On the contrary, EPF is performed for HSIC in each layer of GP or LP, and there are no weights connecting between layers.
6) *Parameter Tuning:* CNN requires tuning a larger number of parameters. By contrast, IGLPN only has one parameter to terminate its iterative process and a set of parameters used to specify GF in (9) and (10), all of which are selected empirically not by training samples. In addition, the number of layers required by IGLPN is determined by an automatic stopping rule compared to the number of iterations in CNN, which is determined empirically.

C. Comparison with CNN

The traditional CNN-based HSI classification consists of three elements: CL, PL, and the FCC layer. The

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

backpropagation learning is used to train the network to adjust the weight between layers. This process is generally time-consuming due to its generating 3-D patches and training of the network. The EPF-based IGLPN can address this issue since it can reduce the input data cube to a single classification map for each class.

As shown in [38], a GF can emulate the human vision system. In this case, we utilize GFs to replace the CL in the CNN to capture human vision features. Applying a GF with n_ϕ different orientations generates n_ϕ GFSMaps, each of which represents one particular orientation. If such GFs are directly applied to the original data cube, high computational complexity will be significantly increased. To mitigate this dilemma, PCA is used as a preprocessing to reduce the data dimensionality prior to data processing, a task that is not used in CNN. Finally, the backpropagation process used by CNN is replaced with an iterative process where SVM is implemented in each iteration to classify the generated GFSMaps for final classification and determine whether an iterative process should be terminated. It should be noted that there are two classifiers are used in IGLPN. The first classifier is EPF-G-g that is used to produce EFP-G-g classification maps. Then, GFs are applied to the EPF-G-g classification maps to generate GFSMaps. The second classifier is SVM that uses its classification maps as a stopping rule to determine when IGLPN must be terminated.

D. Comparison With Current Pyramidal Approaches

While various pyramid-related methods reviewed in Section II may seem similar to IGLPN, the truth of the matter is that many of these pyramid-based CNN methods primarily manipulate the structure of PL or CL of CNN, such as SPP [24] and PyConv [25]. However, these strategies, although effective, are still limited to their basic convolutional-pooling structure, which comes with intrinsic challenges, for example, high computational demands, a considerable parameter count, and limited performance with a small size of training samples.

IGLPN diverges from these pyramid-based CNN methods in three significant aspects. First, IGLPN adopts GLP to replace the (PL, CL) used in the traditional CNN to largely reduce the computational burden of adjusting learnable parameters in CL. Second, IGLPN implements an iterative feedback loop to capture the spatial correlation from classified data samples, so as to improve classification accuracy, a feature that has never been explored by any pyramid-based CNNs. Third, it replaces the FCC layer in CNN with an EPF (SVM-guided filter) classifier. This eliminates the need to adjust learning parameters (weights and biases) in traditional FCC.

V. IGLPN Algorithms

This section is devoted to details of developing three specific IGLPN algorithms for HSIC.

One great benefit gained from IGLPN is to use LP to enhance data samples nearby class boundaries that are often misclassified, thereby improving the classification accuracy of class edges. To fully explore the advantages of IGLPN, three versions of IGLPN are derived, IGPN, ILPN, and IGLPN, each of which has its own merit and will be studied comprehensively by the experiments conducted in Section VII.

Since different layers of GP have varying sizes, each layer except its bottom (i.e., first) layer must be resized as the same size as the original data cube so that the obtained GFSMaps can be added to the original data cube to create a new augmented data cube for the next iteration. To accomplish this task, each layer of GP is interpolated by using the nearest neighbors (NNs) proposed in [36] until the spatial size matches that of the original data cube, as shown in Fig. 3.

Upon implementing IGLPN, four key stages are involved to design IGLPN.

Algorithm 1 (IGLPN):

1) *Initial Condition:* Let $\Omega^{(0)} = \text{HSI}$ and a training sample number of the jth class denoted as n_j where M is the number of classes and $j = 1, \ldots, M$. The initial randomly selected training sample set, $S^{(0)}$, generated according to n_j. Set $k = 1$.
2) *First Stage:*
   a) At the kth iteration, PCA is applied to reduce data dimensionality to produce EPF-G-g maps for different classes. Specifically, the original HSI, $\Omega^{(k)}$, is reduced by PCA to produce the new image cube $\Omega^{(k)}_{\text{PCA}}$. Randomly select a set of training samples, $S^{(k)}$, to implement EPF-G-g to produce M binary maps $\{B^{(k)}_{\text{EPF-G-g},j}\}_{j=1}^{M}$. Specifically, the value of the pixel (x, y) in the jth binary map is defined as follows:
   $$B^{(k)}_{\text{EPF-G-g},j} = \begin{cases} 1, & \text{if pixel at}(x, y) \in \text{class } j \\ 0, & \text{otherwise.} \end{cases} \tag{11}$$
   b) GFs are applied with n_ϕ different orientations to each of the EPF-G-g maps to generate n_ϕ GFSMaps, $\{\text{GFSMap}^{(k)}_j\}_{j=1}^{M}$, which capture n_ϕ feature maps, each of which represents a particular orientation of the human visual system. To remove the dimensional redundancy, we followed an approach in [36] by applying PCA to n_ϕ $\text{GFSMap}^{(k)}_j$ to yield a $\text{GFSMap}^{(k)}_{\text{PCA},j}$ map for the jth class.
3) *Second Stage:*
   a) Three versions, IGPN(m) using the first m layers of GP, ILPN(n) using the first n layers of LP, and IGLPN(p, q) using the first p layers of GP and the first q layers of LP to $\{\text{GFSMap}^{(k)}_{\text{PCA},j}\}_{j=1}^{M}$ for generating soft probability maps defined by $\{\text{S}^{(k)}_{\text{SF},j}\}_{j=1}^{M}$.
   b) Apply NN interpolation to $\{\text{S}^{(k)}_{\text{SF},j}\}_{j=1}^{M}$ for matching the maps' spatial size to the original data cube. The resized maps are defined by $\{\text{S}^{(k)\uparrow}_{\text{SF},j}\}_{j=1}^{M}$.
4) *Third Stage:*
   a) At the kth iteration, there are m $\text{S}^{\uparrow}_{\text{SF}}$ for IGPN(m), n $\text{S}^{\uparrow}_{\text{SF}}$ for ILPN(n), and (p, q) $\text{S}^{\uparrow}_{\text{SF}}$ for IGLPN(p, q) to be fed back and added to the current data cube to create a new augmented data cube. The same process is then repeated over again iteratively.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

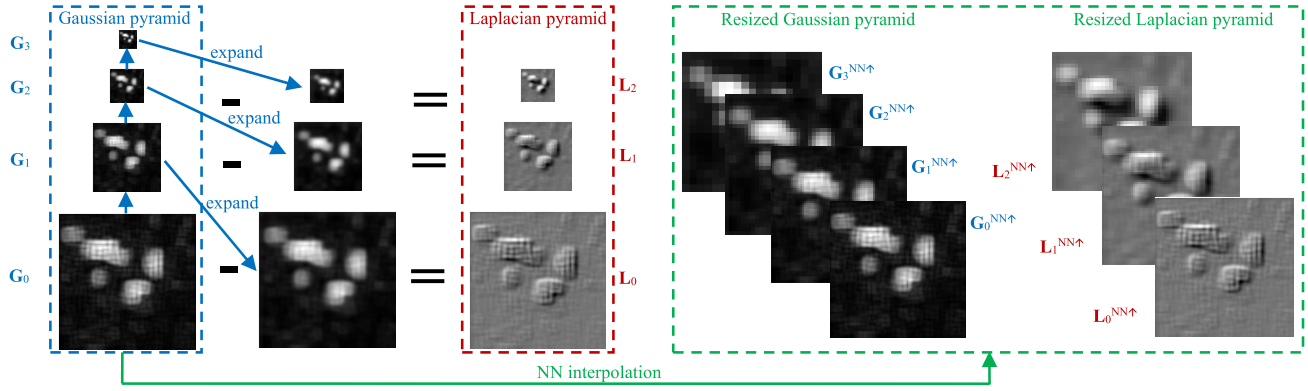


Fig. 3. Schematic of GP and LP, and resizing each layer of GP/LP.

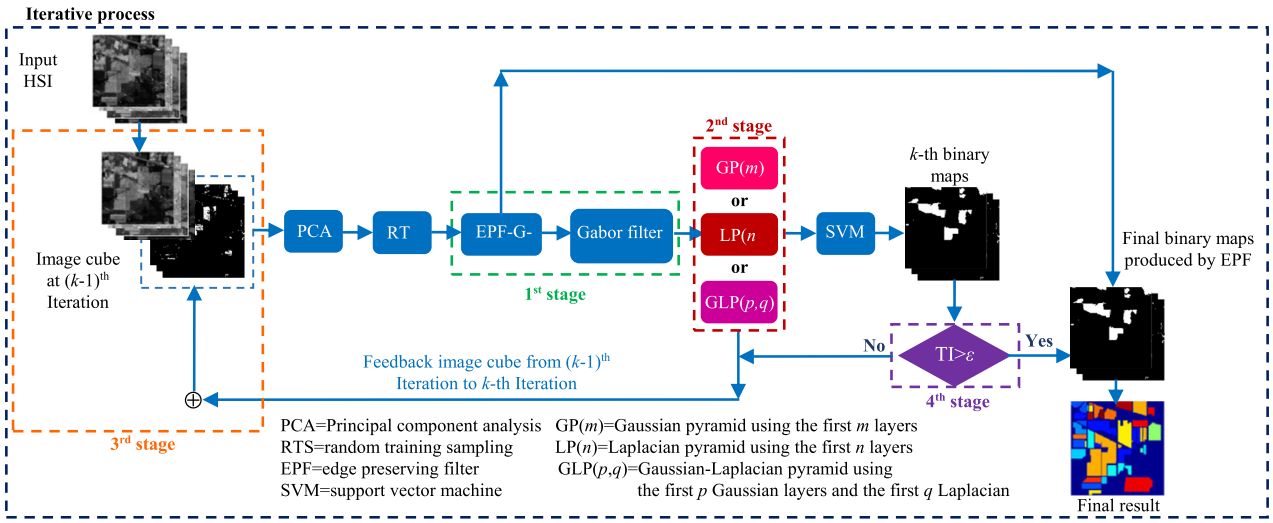


Fig. 4. Graphic diagram of implementing IGLPN at the kth iteration.

b) Generate a new data cube by feeding back the latest k iterations $\{S_{\mathrm{SF},j}^{(k)\uparrow}\}_{j=1}^{M}$

$$\Omega^{(k)} = \Omega^{(k-1)} \cup \left\{ S_{\mathrm{SF},j}^{(k)\uparrow} \right\}_{j=1}^{M}. \tag{12}$$

5) *Fourth Stage (Termination Stage):*

a) Implement SVM with $S^{(k)}$ on $\{S_{\mathrm{SF},j}^{(k)\uparrow}\}_{j=1}^{M}$ to produce the classification map i.e., $\{B_{\mathrm{SVM},j}^{(k)}\}_{j=1}^{M}$ for the stopping rule to determine if the algorithm needs to be terminated or not. The value of the pixel (x, y) in the jth binary map is defined as follows:

$$B_{\mathrm{SVM},j}^{(k)} = \begin{cases} 1, & \text{if pixel at}(x, y) \in \text{class } j \\ 0, & \text{otherwise.} \end{cases} \tag{13}$$

b) Check whether $\{B_{\mathrm{SVM},j}^{(k)}\}_{j=1}^{M}$ satisfies the stopping rule

$$\mathrm{TI}^{(k)} = \frac{1}{M} \sum_{j=1}^{M} \frac{\left| B_{\mathrm{SVM},j}^{(k-1)} \cap B_{\mathrm{SVM},j}^{(k)} \right|}{\left| B_{\mathrm{SVM},j}^{(k-1)} \cup B_{\mathrm{SVM},j}^{(k)} \right|} > \varepsilon \tag{14}$$

where $\mathrm{TI}^{(k)}$ is the Tanimoto index (TI) [41] for the kth iteration.

Fig. 4 depicts the detailed implementation of IGLPN at the kth iteration where the four key stages involved in designing IGLPN are blocks highlighted by dash lines with different colors where full names of abbreviations are provided at the bottom of the block diagram. In particular, it is worth noting from Fig. 4 that the TI specified by (14) in the fourth stage is a criterion used to quantify the similarity between two successive binary classification results of the current and preceding iterations. It does not require GT to terminate the iterative process implemented in IGLPN.

VI. Images To Be Used for Experiments

Four popular real HSIs are described and were used for experiments.

A. Purdue Indiana Indian Pines

The first one is the Purdue Indiana Indian Pines test site with an aerial view shown in Fig. 5(a) along with its GT of 17 class maps in Fig. 5(b) and their class labels in Fig. 5(c). Table I tabulates the number of data samples in each class where there are four small classes with less than 100 samples, classes 9, 7, 1, and 16, and three classes with more than 1000 samples, classes, 2, 11, and 14. Thus, this scene clearly

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

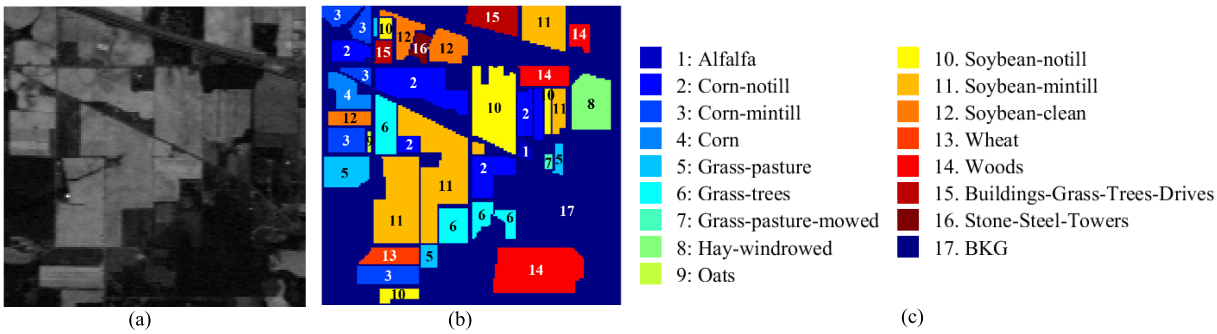


Fig. 5. Purdue's data and its GT with 16 classes. (a) Band 186 (2162.56 nm). (b) GT map. (c) Classes by colors.

TABLE I
CLASS LABELS OF PURDUE E'S DATA

Class 1 (46)	Alfalfa	Class 7 (28)	Grass-pasture-mowed	Class 13 (205)	Wheat
Class 2 (1428)	Corn-notill	Class 8 (478)	Hay-windrowed	Class 14 (1265)	Woods
Class 3 (830)	Corn-mintill	Class 9 (20)	Oats	Class 15 (386)	Bldg-Grass-Trees-Drives
Class 4 (237)	Corn	Class 10 (972)	Soybean-notill	Class 16 (93)	Stone-Steel-Towers
Class 5 (483)	Grass-pasture	Class 11 (2455)	Soybean-mintill	Class 17 (10776)	BKG
Class 6 (730)	Grass-trees	Class 12 (593)	Soybean-clean		

TABLE II
CLASS LABELS OF CLASSES IN SALINAS

Class 1 (2009)	Brocoli green weeds 1	Class 10 (3278)	Corn senesced green weeds
Class 2 (3726)	Brocoli green weeds 2	Class 11 (1068)	Lettuce romaine 4 weeks
Class 3 (1976)	Fallow	Class 12 (1927)	Lettuce romaine 5 weeks
Class 4 (1394)	Fallow rough plow	Class 13 (916)	Lettuce romaine 6 weeks
Class 5 (2678)	Fallow smooth	Class 14 (1070)	Lettuce romaine 7 weeks
Class 6 (3959)	Stubble	Class 15 (7268)	Vinyard untrained
Class 7 (3579)	Celery	Class 16 (1807)	Vinyard vertical trellis
Class 8 (11271)	Grapes untrained	Class 17 (56975)	BKG
Class 9 (6203)	Soil vineyard develop		

has an imbalanced class issue in classification. It is an airborne visible/infrared imaging spectrometer (AVIRIS) image scene and has a size of 145 × 145 × 200 pixel vectors with water absorption bands (bands 104–108, 150–163, and 200). Thus, a total of 220 bands were used for experiments. It should be noted that in many reports, 200 bands were used by excluding water absorption bands such as [5].

B. Salinas

The second image dataset is Salinas in Fig. 6(a), which is also an AVIRIS scene. It was collected over Salinas Valley, CA, USA, with a spatial resolution of 3.7 m per pixel with a spectral resolution of 10 nm. It has a size of 512 × 217 × 224 with 20 water absorption bands: 108–112, 154–167, and 224. Thus, a total of 204 bands were used for experiments. Fig. 6(b) and (c) shows the color composite of the Salinas image along with the corresponding GT class labels. Unlike the Purdue data, the Salinas scene does not have an issue with imbalanced classes. According to the GT tabulated in Table II, the smallest class is class 13 with 916 data samples.

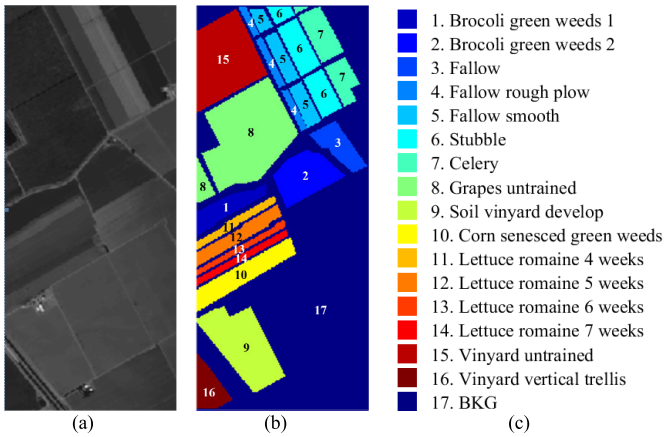


Fig. 6. Salinas and its GT with 16 classes. (a) Salinas scene. (b) GT image. (c) Classes by colors.

C. University of Pavia

The third HSI dataset was the University of Pavia image shown in Fig. 7, which is an urban area surrounding the University of Pavia, Pavia, Italy. It was recorded by the ROSIS-03 satellite sensor over an urban area surrounding the University of Pavia. It is of size 610 × 340 × 115 with a spatial resolution of 1.3 m per pixel and a spectral coverage ranging from 0.43 to 0.86 μm with a spectral resolution of 4 nm (12 most noisy channels were removed before experiments). Fig. 7(b) provides its GT map of nine classes along with color class labels in Fig. 7(c). Table III also tabulates the number of data samples in parentheses collected for each class. Like the Salinas scene, this scene also has very large classes with

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

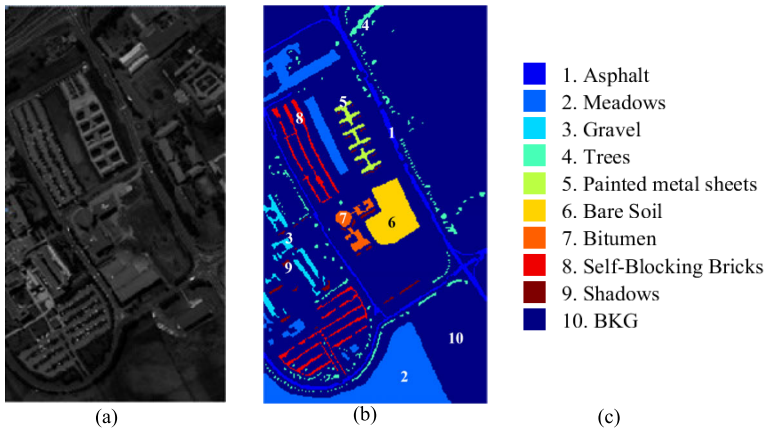


Fig. 7. University of Pavia and its GT with nine classes. (a) University of Pavia scene. (b) GT map. (c) Classes by colors.

TABLE III
CLASS LABELS OF THE UNIVERSITY OF PAVIA

Class 1 (6631)	Asphalt	Class 5 (1345)	Painted metal sheets	Class 9 (947)	Shadows
Class 2 (18649)	Meadows	Class 6 (5029)	Bare Soil	Class 10 (164624)	BKG
Class 3 (2099)	Gravel	Class 7 (1330)	Bitumen		
Class 4 (3064)	Trees	Class 8 (3682)	Self-Blocking Bricks		

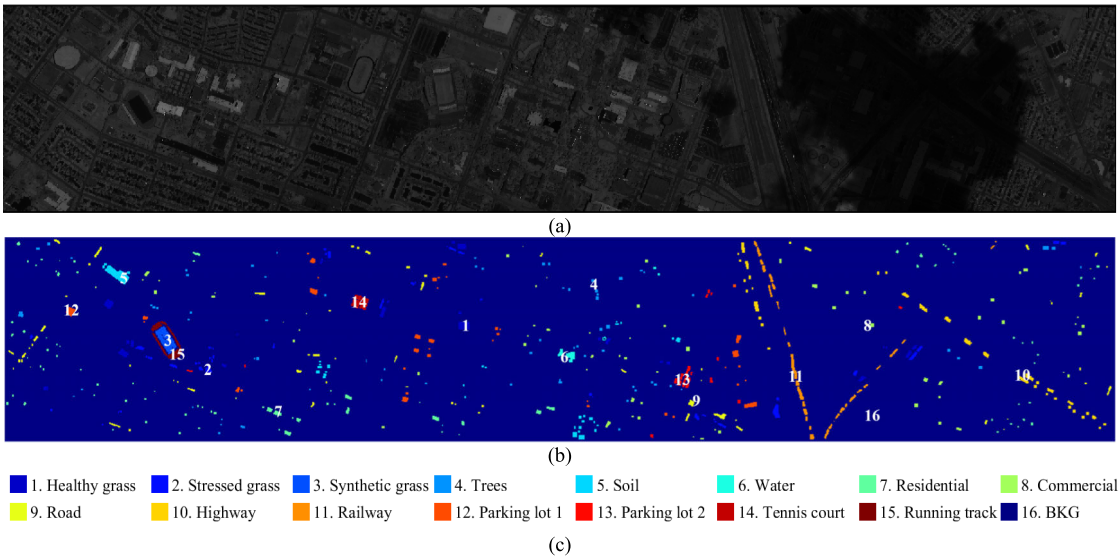


Fig. 8. Houston data and its GT with 15 classes. (a) Houston data. (b) Ground truth. (c) class labels.

only one small class with less than 1000 data samples and class 9 with 947 samples. However, this scene has a more complicated BKG than the other two studied scenes as already shown in [5] where the precision rate of this scene was much lower than the other two scenes. It should be noted that a total of 103 bands were used for experiments.

D. Houston Data

The fourth and final image scene is the Houston dataset, collected over the campus area of the University of Houston and its neighboring urban areas, as shown in Fig. 8. The image was captured by a compact airborne spectrographic image (CASI) sensor in 2012. This dataset was made public in the 2013 IEEE GRSS Data Fusion Contest. It has dimensions of 349 × 1905 × 144, with 144 spectral bands ranging from 380 to 1050 nm and a spatial resolution of 2.5 m. Table IV provides a summary of the number of data samples, indicated in parentheses, collected for each class.

E. Classification Measures

From a statistical signal processing viewpoint [42, Sec. V], classification can be performed by two approaches: a priori classification with background (BKG) removed by GT and a posteriori classification with BKG included.

1) A Priori Classification Criteria With BKG Removed: Assume that M is the number of classes, $\{C_j\}_{j=1}^{M}$, n_m is the number of data samples in C_m, and N is the total number of data samples, $N = \sum_{m=1}^{M} n_m$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
CLASS LABELS OF HOUSTON'S DATA

Class 1 (1251)	Healthy grass	Class 7 (1268)	Residential	Class 13 (469)	Parking lot 2
Class 2 (1254)	Stressed grass	Class 8 (1244)	Commercial	Class 14 (428)	Tennis court
Class 3 (697)	Synthetic grass	Class 9 (1252)	Road	Class 15 (660)	Running track
Class 4 (1244)	Tress	Class 10 (1227)	Highway	Class 16 (649816)	BKG
Class 5 (1242)	Soil	Class 11 (1235)	Railway		
Class 6 (325)	Water	Class 12 (1233)	Parking lot 1		

In traditional HSIC, the popular performance measurements are class accuracy (A) and overall accuracy (OA), given by

$$p_{\mathrm{A}}(C_m) = \text{accuracy of the } m^{\text{th}} \text{ class} = \frac{n_{mm}}{\sum_{j=1}^{M} n_{jm}} = \frac{n_{mm}}{n_m} \tag{15}$$

and

$$\mathrm{P_{OA}} = \frac{1}{N}\sum_{m=1}^{M} n_{mm} = \sum_{m=1}^{M} \frac{n_m}{N} p_{\mathrm{A}}(C_m) \tag{16}$$

where (n_m/N) is the sample ratio of the mth class C_m. If we assume the probability of the mth class C_m to be equally likely by letting $(n_m/N) = (1/M)$, then we can further define average accuracy (AA), $\mathrm{P_{AA}}$, as

$$\mathrm{P_{AA}} = \frac{1}{M}\sum_{m=1}^{M} \frac{n_{mm}}{n_m} = \frac{1}{M}\sum_{m=1}^{M} p_{\mathrm{A}}(C_m). \tag{17}$$

The key difference between $\mathrm{P_{OA}}$ and $\mathrm{P_{AA}}$ lies in the fact that $\mathrm{P_{OA}}$ weights the accuracy of each class according to its sample size, while $\mathrm{P_{AA}}$ treats each class as equally important. As a result, $\mathrm{P_{OA}}$ is generally dominated by the accuracies of large classes at the expense of the accuracies of small classes, while $\mathrm{P_{AA}}$ is not. Consequently, $\mathrm{P_{AA}}$ is more appropriate to deal with the imbalanced class issue. It should be noted that OA is generally known as producer's accuracy.

2) A Posteriori Classification Criteria With BKG Included: The reason that $\mathrm{P_{OA}}$ in (16) and $\mathrm{P_{AA}}$ in (17) are referred to as a priori criteria is because it assumes that the complete GT is provided a priori so that the classification is performed on the GT data samples with known BKG removed. This common practice has been widely used in HSIC in the literature. However, in many real scenarios, the complete GT may not be available, and evaluating classification performance without BKG is not realistic. In this case, we need to factor BKG in the classification evaluation. To cope with the issue, instead of a priori classification that calculates $\mathrm{P_{OA}}$ and $\mathrm{P_{AA}}$ for GT$\{x_n\}_{n=1}^{N}$, a posteriori classification is performed on the classified data samples, denoted by $\{\hat{x}_n\}_{n=1}^{N}$ according to classification maps, $\{\hat{C}_j\}_{j=1}^{M}$ produced by a classifier δ, in which case BKG is included for classification. In other words, a posteriori classification is carried out by designing an a posteriori classifier $\hat{\delta}$ that assigns a classified data sample $\hat{x}$ in $\hat{C}_m$ to a class C_m to which it is supposed to belong according to GT.

According to Chang et al. [5], we can further define a posteriori classification measures, $\mathrm{P_{PR}}(\hat{C}_j)$, and the precision rate, $\mathrm{P_{PR}}$, corresponding to a priori classification measures, $\mathrm{P_A}$ in (17) and $\mathrm{P_{OA}}$ in (18), as follows.

$$\mathrm{P_{PR}}(\hat{C}_j) = \sum_{\mathbf{x}\in C_j, \hat{\mathbf{x}}\in\hat{C}_j} p(\mathbf{x}|\hat{\mathbf{x}}) = \frac{\hat{n}_{jj}}{\sum_{i=1}^{M}\hat{n}_{ji}} \tag{18}$$

where $\hat{n}_{ji}$ is the number of data samples classified in the jth class, C_j, which are supposed to be in the ith class, C_i, and

$$\mathrm{P_{PR}} = \sum_{i=1}^{M} p(\hat{C}_i)\mathrm{P_{PR}}(\hat{C}_i) \tag{19}$$

where $p(\hat{C}_i) = (\hat{n}_i/N) =$ a posteriori probability of $\hat{C}_i$.

Thus, by virtue of (18) and (19), we can define the average PR (APR), $\mathrm{P_{APR}}$, in correspondence to $\mathrm{P_{AA}}$ in (17) as

$$\mathrm{P_{APR}} = \frac{1}{M}\sum_{m=1}^{M}\mathrm{P_{PR}}(\hat{C}_m). \tag{20}$$

Since n_{mm} is identical to $\hat{n}_{mm}$ and $\hat{N} = \sum_{i=1}^{M}\hat{n}_i$ equal to $N = \sum_{i=1}^{M} n_i$. As a result, an overall PR (OPR), $\mathrm{P_{OPR}}$, in correspondence to $\mathrm{P_{OA}}$ in (16) can be further defined as

$$\mathrm{P_{OPR}} = \sum_{i=1}^{M}\left(\frac{\hat{n}_i}{\hat{N}}\right)\mathrm{P_{PR}}(\hat{C}_i). \tag{21}$$

Interestingly, if we remove BKG from classification, then

$$\mathrm{P_{OPR}} = \sum_{i=1}^{M}\left(\frac{\hat{n}_i}{\hat{N}}\right)\mathrm{P_{PR}}(\hat{C}_i) = \sum_{i=1}^{M}\left(\frac{n_i}{N}\right)\mathrm{P_A}(C_i) = \mathrm{P_{OA}} \tag{22}$$

which implies that $\mathrm{P_{OPR}}$ is essentially the same $\mathrm{P_{OA}}$.

It should be noted that PR is also known as user's accuracy to address BKG issues and is the one that can include all data samples for evaluation. For those who are interested in how to calculate OA, AA, and PR, we refer details to the illustrative examples presented in [42, Sec. VI].

VII. EXPERIMENTAL RESULTS AND DISCUSSION

This section conducted experiments to evaluate the performance of our proposed methods, IGPN(m), ILPN(n), IGLPN(p, q), and IEPF-Gabor compared to four recently developed approaches: IEPF-G-g [34], IRTS-3DCNN [5], IRTS-EPF-G-g [43], and PCAWSD [28], all of which are summarized in Table V.

It should be noted that, in [5], IRTS-3DCNN was compared and shown to perform better than several recently developed CNN-based classification methods. In this case, only IRTS-3DCNN was used for comparison.

Since IGLPN implements PCA as a preprocessing shown in Fig. 4, two ways to determine the number of PCs, n_{PC}, were adopted. One is empirical determination according to

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EIGHT TEST METHODS COMPARED FOR EXPERIMENTS

Method	Details
IEPF-Gabor	iterative EPF using Gabor filter as a spatial filter
IEPF-G-g [34]	iterative EPF-G-g using a guided filter and the 1st PC
IGPN(m)	IGPN using PCA, Gabor filter and the first m layers of Gaussian pyramid.
ILPN(n)	ILPN using PCA, Gabor filter and the first n layers of Laplacian pyramid.
IGLPN(p,q)	IGLPN using PCA, Gabor filter and the first p layers of Gaussian pyramid as well as the first q layers of Laplacian pyramid.
IRTS-3DCNN [5]	iterative random training sampling 3D CNNN
IRTS-EPF-G-g [43]	iterative random training sampling EPF-G-g
PCAWSD [28]	pyramid coordinate attention and weighted self-distillation

[8, Supplementary Material]. The other is VD [44], [45], [46]. Also, given the fact that IGLPN makes use of pyramids to acquire multiscale features, we implemented PCAWSD as a benchmark to compare our proposed methods. In addition, IGLPN is proposed as an alternative to CNN. Thus, IRTS-3DCNN is also implemented as another benchmark to demonstrate the performance of IGLPN against the traditional CNN structure.

A. Parameter Settings

Generally, how to select a training sample pool has a significant impact on classification performance. There are two factors involved in the selection of training samples. One is the number of training samples. The other is the training sample size for each class.

A commonly used approach is to determine the size of training samples for each class according to the sample ratio. Unfortunately, such selection will result in the sacrifice of small classes whose classification accuracies may be dominated by the accuracies of large classes. To address this issue, one was developed by Zhong et al. [47], which used a class feature-based method to select a training sample size for each class. Another one was developed by Kang [48], which is available on the website. In this article, Kang's method was used to determine the training sample size for each class listed in Table VI, and 1% of data samples were randomly selected for experiments.

The parameters used the GFs specified in Table VII for the four datasets where the values were determined empirically.

It is worth noting that IGLPN used four parameters in Table VII to specify GFs and one parameter, the TI threshold value used in (14) as a stopping criterion. All these parameters are selected empirically compared to DL-based HSIC methods, which require a large number of parameters to be tuned by training samples. For example, a typical DL model often requires tuning a large parameter set, such as filter size, patch size, batch size, stride, pooling size, the number of neurons, learning rate, epochs, and the number of convolution blocks. By contrast, IGLPN does not require a backward propagation used by CNN to adjust a large set of learnable parameters, such as the weight of the convolutional kernel. IGLPN replaces it with an IFP with no need for weights but rather one parameter, TI to terminate the process.

TABLE VI
1% TRAINING SAMPLE SIZES FOR PURDUE DATA, SALINAS, AND U. OF PAVIA

Class	Purdue data	Salinas	U. of Pavia
1	6	35	48
2	6	33	51
3	6	34	47
4	6	33	47
5	7	34	47
6	7	33	47
7	6	33	47
8	6	35	47
9	6	34	47
10	6	36	
11	7	33	
12	7	33	
13	6	33	
14	7	33	
15	7	36	
16	6	33	
Total	102	541	428

TABLE VII
PARAMETER SETTINGS FOR PURDUE DATA, SALINAS, U. OF PAVIA, AND HOUSTON

Parameters	Purdue data	Salinas	U. of Pavia	Houston
Orientations ϕ	45°, 90°, 135°, 180°	0°, 45°, 90°, 135°, 180°	0°, 45°, 90°, 135°, 180°	45°, 90°, 135°, 180°
Wavelengths λ	4, 8	4, 8	4, 8	4, 8
Bandwidth b	2.5	2.5	2.5	2.5
Spatial aspect ratio γ	0.5	0.5	0.5	0.5

B. Purdue Indiana Indian Pines

For visual inspection, Fig. 9(a) shows the classification maps without BKG and with BKG produced by the 13 test methods, IEPF-G-g, IRTS-EPF-G-g, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IGPN(3), ILPN(4), IGLPN(3, 2), IEPF-Gabor-VD, IGPN-VD(3), ILPN-VD(4), and IGLPN-VD(3, 2) with n_{PC}, determined empirically in [8, Supplementary Material] and determined by VD [49], respectively. As shown in Fig. 9(a), when BKG was removed from classification, all the test classifiers performed reasonably well. However, if we paid attention to the classification of the rectangle area of a yellow class 10 located at the upper left corner highlighted by a yellow open circle in the GT map, only IRTS-EPF-G-g, IRTS-3DCNN, IEPF-Gabor, IGPN(3), and IGLPN-VD(3, 2) performed well, specifically, IGPN(3) and IGLPN-VD(3, 2) among the best. Now, if we looked into the classification maps in Fig. 9(b), which included BKG in the classification, the results were very poor. As can be seen in Fig. 9(b), the classified data samples in all the classes were heavily mixed with BKG samples. Such mixing activities were avoided and eliminated when BKG was removed from the classification. Specifically, for the same yellow-highlighted rectangle area of class 10 in the GT map, only IGPN(3) and IGLPN-VD(3, 2) could classify this area. By visual inspection, IGLPN-VD(3, 2) seemed to be the best according to Fig. 9(a) and (b).

Fig. 9(a) and (b) only provides a qualitative comparison. To conduct quantitative analysis, Table VIII tabulates the results of a priori criteria $\mathrm{P_A}$, $\mathrm{P_{OA}}$, $\mathrm{P_{AA}}$ with BKG removed and a posteriori criteria, $\mathrm{P_{APR}}$ and $\mathrm{P_{OPR}}$ with BKG included calculated from Fig. 9(a) and (b) where the best results

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII

P_A, P_{OA}, P_{AA}, P_{PR}, P_{OPR}, AND P_{APR} PRODUCED BY IEPF-G-G, IRTS-EPF-G-G, PCAWSD, IRTS-3DCNN, IEPF-GABOR, IGPN(3), ILPN(4), AND IGLPN(3, 2) FOR PURDUE DATA (IGPN, ILPN, AND IGLPN WITH $n_{PC} = 30$ DETERMINED EMPIRICALLY AND $n_{PC} = 18$ DETERMINED VD)

	IEPF-G-g	IRTS-EPF-G-g	GFDN	PCA-WSD	IRTS-3DCNN	IEPF-Gabor	IGPN(3)	ILPN(4)	IGLPN (3,2)	IEPF-Gabor--VD	IGPN-VD(3)	ILPN-VD(4)	IGLPN-VD(3,2)
n_{PC}						30	30	30	30	18	18	18	18
Class	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$
1	**100**	98.55	99.28	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**
2	53.01	67.25	53.13	74.14	86.69	79.81	**89.78**	80.2	81.26	88.19	86.56	79.74	88.87
3	68.07	80.28	65.38	73.73	90.08	89.4	89.8	82.01	93.61	83.01	**93.69**	77.47	80.44
4	82.7	85.23	98.59	95.22	99.72	99.86	99.72	94.09	99.86	**100**	98.87	88.47	97.19
5	81.09	83.44	80.61	78.68	93.93	91.86	93.03	84.68	**94.9**	90.27	87.71	82.26	86.82
6	90.5	90.14	87.49	94.79	99.09	**99.82**	99.45	94.29	99.27	99.63	98.99	93.56	99.32
7	95.24	96.43	**100**	**100**	**100**	**100**	**100**	**100**	**100**	98.81	**100**	**100**	**100**
8	76.64	94.98	95.26	99.93	99.93	**100**	**100**	94.56	**100**	**100**	**100**	**100**	99.72
9	**100**	96.67	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**
10	71.4	82.68	80.32	72.56	84.81	89.47	86.49	76.85	90.54	85.53	87.55	81.31	**92.9**
11	55.59	73.2	71.28	69.06	71.46	79.93	**88.05**	73.63	79.68	79.04	83.82	66.33	79.9
12	65.65	82.97	64.81	76.95	85.72	92.13	**95.34**	88.2	94.94	91.91	91.51	75.15	92.47
13	99.51	98.86	98.54	**100**	99.84	99.84	99.67	99.35	99.84	99.67	99.67	99.84	99.51
14	89.46	89.86	87.22	91.68	94.89	96.1	97.95	91.73	97.44	94.91	97.89	86.82	**99.63**
15	77.46	96.98	86.7	94.04	99.14	96.8	99.91	94.21	99.83	99.31	99.91	95.94	**100**
16	96.77	79.21	98.92	**100**	99.64	98.21	98.21	95.7	97.85	98.21	96.06	97.13	98.57
P_{OA}	70.27	81.13	75.7	80.18	87.21	89.05	**92.68**	83.78	90.17	88.95	91.09	80.71	90.13
P_{AA}	81.44	87.3	85.47	88.8	94.06	94.58	**96.09**	90.59	95.56	94.28	95.14	89	94.71
P_{OPR}	34.26	39.55	36.9	39.08	42.51	43.41	**45.18**	40.84	43.95	43.36	44.4	39.34	43.93
P_{APR}	43.51	**57.44**	33.31	37.21	45.05	46.79	47.65	42.89	46.75	45.22	44.5	40.03	42.57
Time	7.26	9.41	15.59	1425.29	1756.33	9.35	12.96	12.32	13.75	7.86	12.89	11.33	13.48

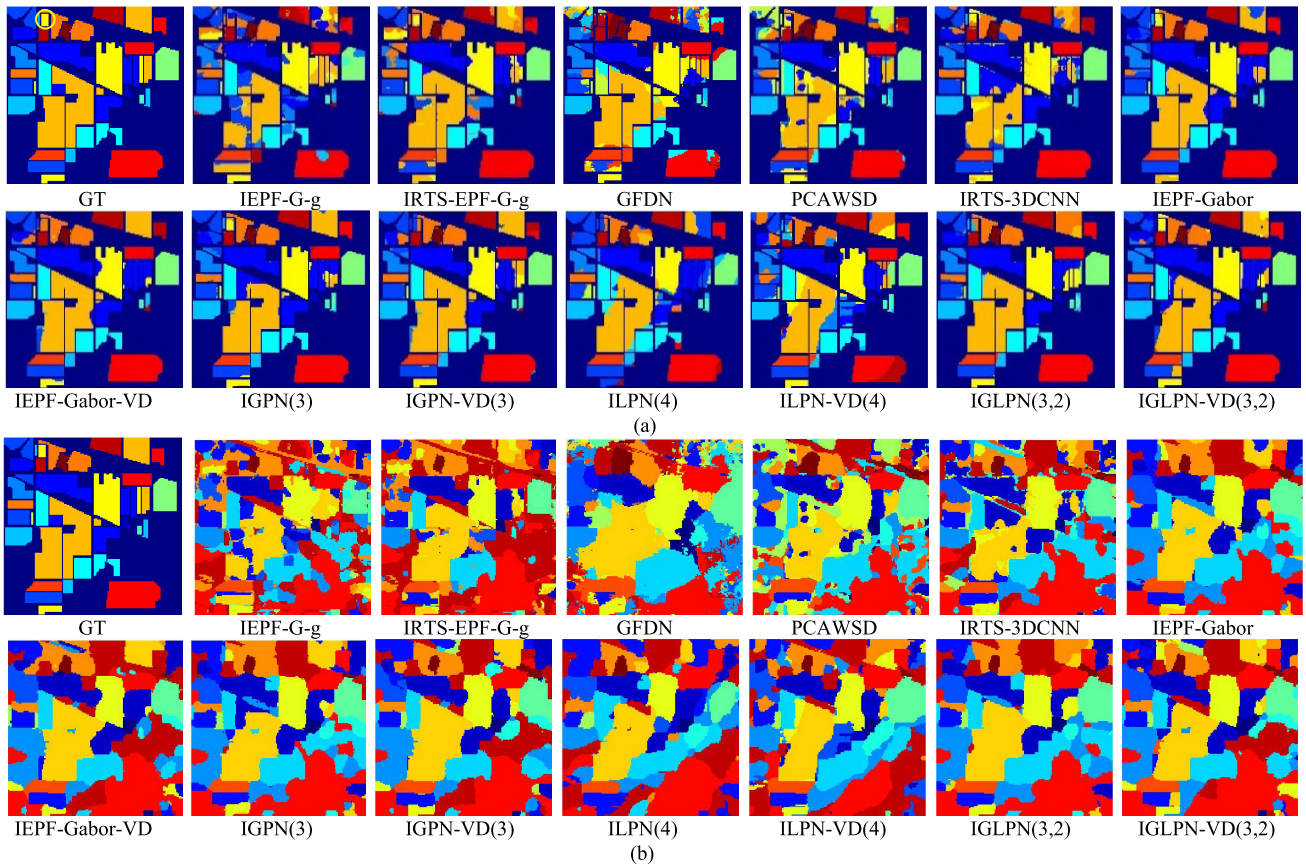


Fig. 9. Classification maps of by GT, IEPF-G-g, IRTS-EPF-G-g, GFDN, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IEPF-Gabor-VD, IGPN(3), IGPN-VD(3), ILPN(4), ILPN-VD(4), IGLPN(3, 2), and IGLPN-VD(3, 2) for the Purdue data using 1% of training samples. (a) Without BKG. (b) With BKG.

are boldfaced and P_{OA} was generally lower than P_{AA}. The experiments were run 5 times.

Among 13 methods, IGPN(3) achieved the best results in terms of P_{OA} and P_{OA}. It is worth noting that with the introduction of GP, the accuracy was improved from $P_{OA}/P_{OA} = 89.05\%/94.58\%$ to $P_{OA}/P_{OA} = 92.68\%/96.09\%$ compared to IEPF-Gabor. However, ILPN(4) did not show any significant improvement in performance compared to

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

TABLE IX

P_A, P_{OA}, P_{AA}, P_{PR}, P_{OPR}, AND P_{APR} PRODUCED BY IEPF-G-G, IRTS-EPF-G-G, PCAWSD, IRTS-3DCNN, IEPF-GABOR, IGPN(7), ILPN(6), AND IGLPN(7, 6) FOR SALINAS DATA (IGPN, ILPN, AND IGLPN WITH $n_{PC} = 15$ DETERMINED EMPIRICALLY AND $n_{PC} = 24$ DETERMINED BY VD)

	IEPF-G-g	IRTS-EPF-G-g	GFDN	PCA-WSD	IRTS-3DCNN	IEPF-Gabor	IGPN(7)	ILPN(6)	IGLPN (7,6)	IEPF-Gabor-VD	IGPN-VD(7)	ILPN-VD(6)	IGLPN-VD(7,6)
n_{PC}						15	15	15	15	24	24	24	24
Class	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$
1	99.68	**100**	99.8	99.88	99.59	**100**	**100**	99.77	**100**	**100**	**100**	**100**	**100**
2	99.27	99.74	99.71	**100**	99.95	**100**	**100**	**100**	**100**	**100**	**100**	**100**	99.56
3	98.84	99.36	98.77	**100**	**100**	**100**	**100**	**100**	99.88	**100**	**100**	98.84	**100**
4	98.83	99.81	98.49	99.42	99.52	99.23	99.91	99.76	99.91	99.57	**99.95**	99.76	99.74
5	98.68	98.88	96.91	99.25	99.33	99.38	98.71	98.7	98.72	**99.52**	99.12	98.48	99.14
6	99.82	99.81	99.82	99.96	**100**	99.99	99.92	99.87	99.95	**100**	**100**	99.59	99.93
7	99.6	99.41	99.24	99.93	99.99	**100**	**100**	**100**	99.99	99.96	99.99	99.51	**100**
8	91.21	97.05	86.28	96.35	87.45	90.63	97	97.77	98.1	93.44	**98.13**	97.92	97.86
9	98.37	98.75	99.15	98.96	99.9	**100**	99.7	99.75	99.93	**100**	99.62	99.81	99.82
10	98.94	**99.89**	97.94	98.04	96.73	99.18	98.56	99.47	99.26	98.86	99.38	99.31	99.73
11	98.13	98.88	99.25	99.66	97.25	**100**	**100**	99.75	99.91	99.97	**100**	**100**	**100**
12	99.95	**100**	98.93	**100**	**100**	**100**	**100**	**100**	**100**	**100**	99.98	**100**	**100**
13	99.23	99.34	99.56	99.85	98.91	99.71	98.76	99.05	99.24	**100**	99.89	99.56	99.85
14	97.79	98.44	97.69	99.72	99.13	99.69	98.54	98.63	98.13	**99.78**	99.66	99.25	99.38
15	92.37	98.54	95.54	96.77	80.39	91.36	99	99.02	99.23	93.69	99.57	98.08	**99.84**
16	98.39	98.23	**100**	99.56	99.67	**100**	**100**	99.89	**100**	**100**	**100**	**100**	**100**
P_{OA}	96.51	98.76	95.89	98.48	94.38	96.78	99	99.2	99.32	97.68	**99.42**	99.04	99.4
P_{AA}	98.07	99.13	97.94	99.21	97.36	98.7	99.38	99.46	99.52	99.05	**99.71**	99.38	99.68
P_{OPR}	47.02	48.12	46.72	47.97	45.98	47.15	48.23	48.33	48.39	47.59	**48.44**	48.25	48.43
P_{APR}	58.13	**60.93**	55.95	46.8	52.97	54.83	55.38	55.59	55.78	56.07	56.5	55.33	56.48
Time	76.06	76.04	22.65	6298.98	2268.18	59.51	107.3	96.31	194.38	43.04	122.12	111.9	151.98

IEPF-Gabor. More interestingly, IGPN(3) significantly outperformed DL methods, GFDN, PCAWSD, and IRTS-3DCNN in terms of accuracy and computation time efficiency.

In the comparison of the difference of n_{PC}, despite the fact that IGLPN using empirically determined n_{PC} performed slightly better than IGLPN using VD-determined n_{PC} by 1.5% for P_{OA} and 0.8% for P_{AA}, VD provided an automatic rule to determine n_{PC} rather than empirically which must be done by trial and error.

As for classification with BKG included, P_{OPR} and P_{APR} were very low below, and most P_{OPR} and P_{APR} values were lower than 40% and 50%, respectively. Specifically, P_{OPR} was lower than P_{APR} because poor PR values of large classes overdominated that of small classes. Comparing Fig. 9(a) and (b), we immediately realized that BKG played a significant role in classification. This leads to two key issues. One is that most of the HSIC techniques reported in the literature, which performed very well with BKG removed from GT, were actually performed very poorly when BKG was included for classification. Another is to explain why most HSIC techniques only perform on data, which removes BKG. If these techniques are applied to data with BKG included, their classification results would be very poor with accuracy below 50%.

C. Salinas

Fig. 10(a) and (b) shows the classification maps without BKG and with BKG produced by the 13 test methods, IEPF-G-g, IRTS-EPF-G-g, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IGPN(3), ILPN(4), IGLPN(3, 2), IEPF-Gabor-VD, IGPN-VD(3), ILPN-VD(4), and IGLPN-VD(3, 2) with n_{PC}, determined empirically in [8, Supplementary Material] and determined by VD [49], respectively. Unlike the Purdue data that have four small classes with less than 100 data samples, all the classes in Salinas are very large with the smallest class size of 912 data samples. Compared to the Purdue data, the classification of these data without BKG in Fig. 10(a) was relatively easy where most of the test classifiers performed reasonably well. The poorest classification class was the area of green class 8 highlighted by a yellow open circle in the GT map. By visual inspection, it seemed that the brown class 15 located above the green class 8 had common misclassified data samples. Even so, IGPN-VD(7) performed the best. Now, if we included BKG in the classification, the classification maps in Fig. 10(b) were also poor. An intriguing finding was the performance of PCAWSD, which performed well on the classification of the light green class 9 at the bottom in Fig. 10(a) but performed the worst in Fig. 10(b). This illustrated that BKG has a tremendous impact on classification performance. By visual inspection of Fig. 10(b), various versions of IGLPN generally performed better than other classifiers.

Table IX tabulates the results of a priori criteria P_A, P_{OA}, and P_{AA} with BKG removed and a posteriori criteria, P_{APR} and P_{OPR} with BKG included calculated from Fig. 10(a) and (b) where the best results are boldfaced and achieved by IGPN-VD(7). The experiments were run five times. Like the Purdue data, P_{OA} was also lower than P_{AA}. As observed from Fig. 10(a) and (b), PCAWSD performed well on Salinas without BKG in Fig. 10(a), but it turned out to be the worst in terms of P_{APR} in Fig. 10(b). Notably, the computational times required by two deep-learning-based PCAWSD and IRTS-3DCNN were more than ten times that of IGLPN.

D. University of Pavia

The BKG of U. of Pavia is more complicated than the Purdue data and Salina. Fig. 11(a) and (b) shows the classification maps without BKG and with BKG produced by the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

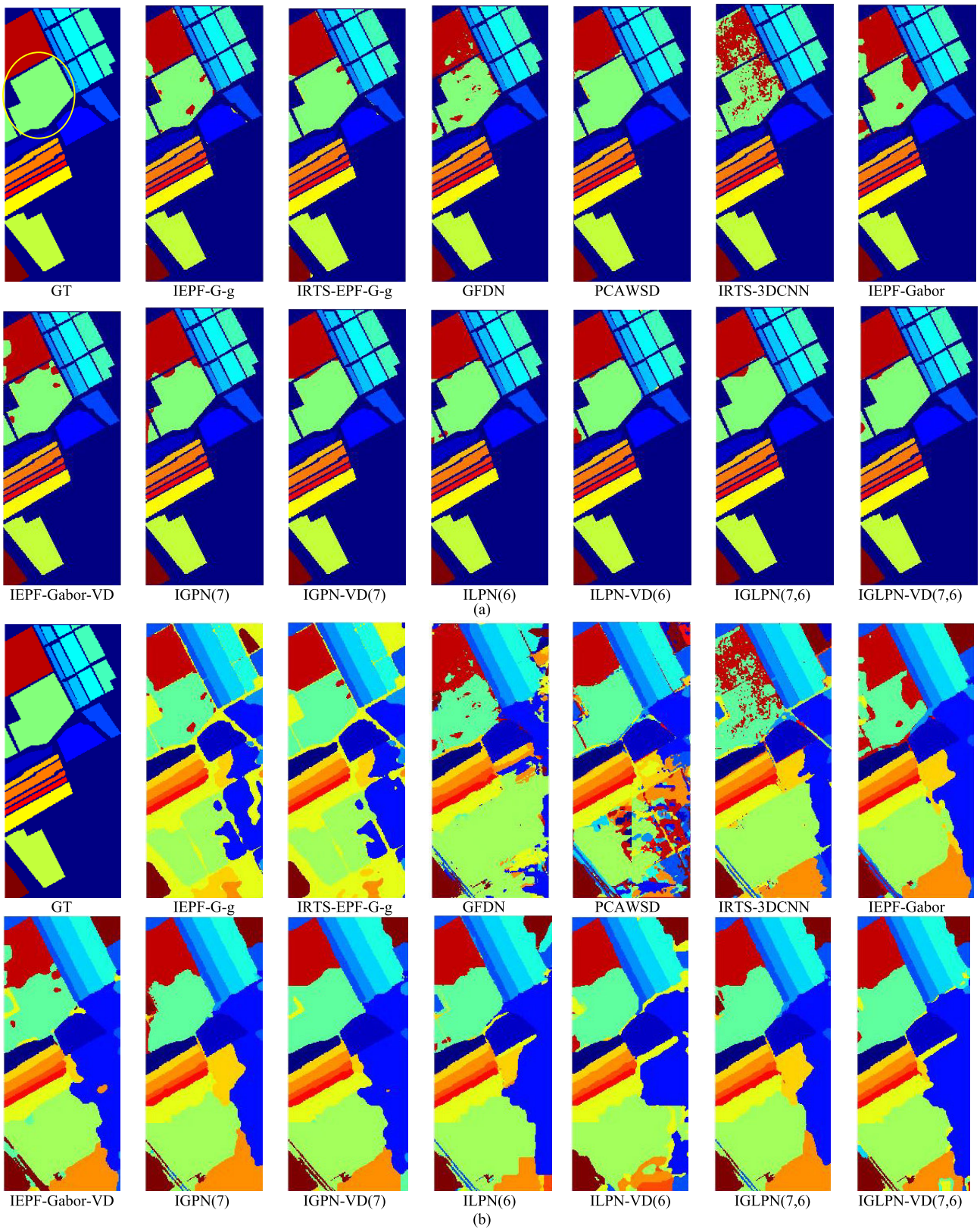


Fig. 10. Classification maps of by GT, IEPF-G-g, IRTS-EPF-G-g, GFDN, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IEPF-Gabor-VD, IGPN(7), IGPN-VD(7), ILPN(6), ILPN-VD(6), IGLPN(7, 6), and IGLPN-VD(7, 6) for the Salinas data using 1% of training samples. (a) Without BKG. (b) With BKG.

13 test methods, IEPF-G-g, IRTS-EPF-G-g, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IGPN(3), ILPN(4), IGLPN(3, 2), IEPF-Gabor-VD, IGPN-VD(3), ILPN-VD(4), and IGLPN-VD(3, 2) with n_{PC}, determined empirically in [8, Supplementary Material] and determined by VD [49], respectively. As shown in Fig. 11(a) with BKG removed, all 13 test classifiers performed

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

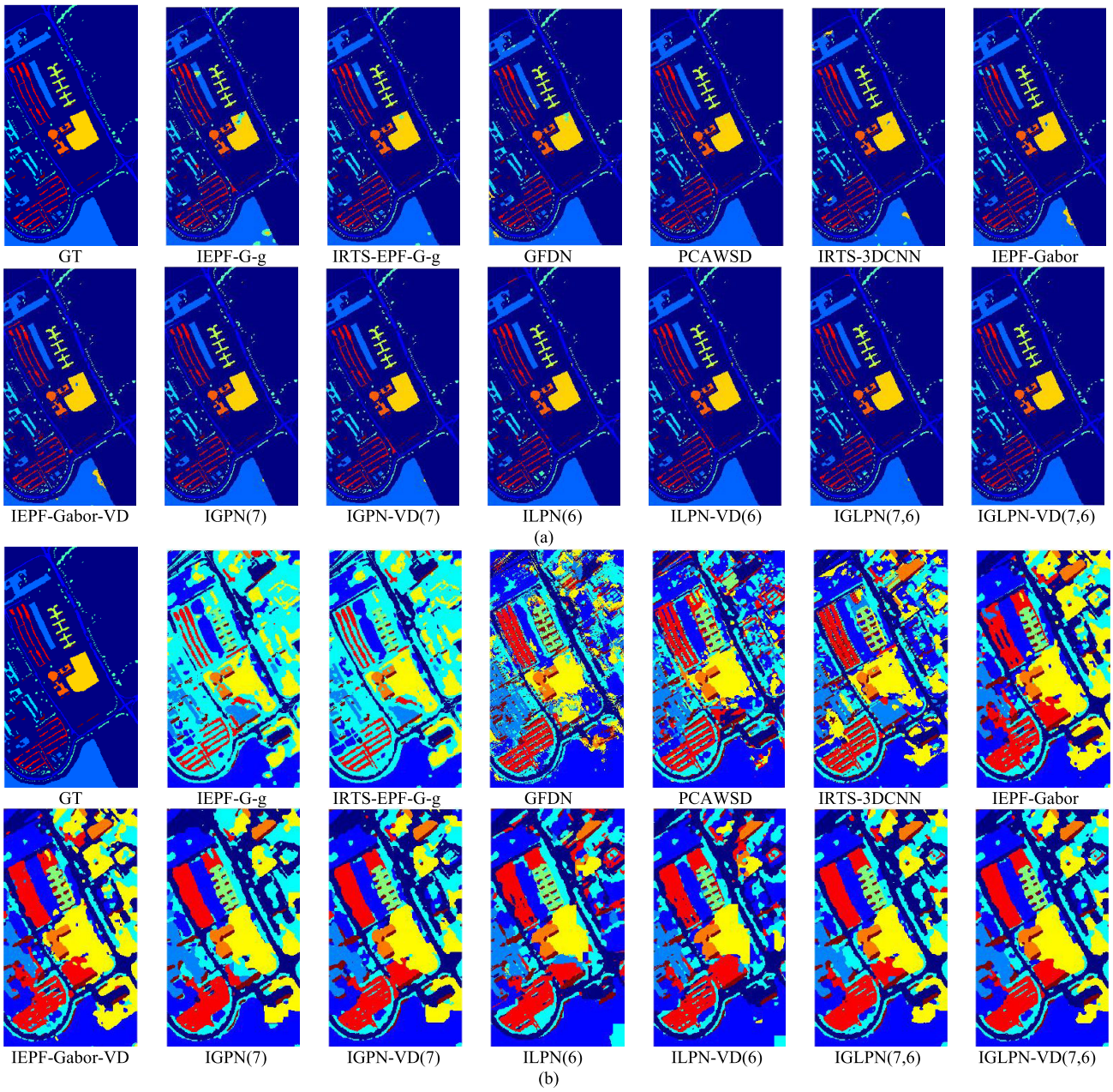


Fig. 11. Classification maps of by GT, IEPF-G-g, IRTS-EPF-G-g, GFDN, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IEPF-Gabor-VD, IGPN(7), IGPN-VD(7), ILPN(6), ILPN-VD(6), IGLPN(7, 6), and IGLPN-VD(7, 6) for the U. of Pavia data using 1% of training samples. (a) Without BKG. (b) With BKG.

well. It is difficult to find which one was the best by visual inspection. However, if we compared the classification maps in Fig. 11(a) without BKG and the classification maps where each class was clearly classified, the classification maps in Fig. 11(b) with BKG included showed that nearly every class was heavily mixed by BKG where the class-BKG mixing was even worse than that in Figs. 9(b) and 10(b). By visual inspection, it seemed that IEPF-G-g, IRTS-EPF-G-g, PCAWSD, and IRTS-3DCNN were among the best.

To further see how BKG interfered with the classification, Table X tabulates the results of a priori criteria, P_A, P_{OA}, and P_{AA}, and a posteriori criteria, P_{OPR} and P_{APR}, from Fig. 11(a) and (b) where the experiments were run five times. It clearly showed that the values of P_{OPR} and P_{APR} for U. of Pavia were very low and even worse than those obtained for the Purdue data and Salinas, and the best results were achieved by IGPN(7). This evidence indicated that the BKG in the U. of Pavia was more complicated than the Purdue data and Salinas. Interestingly, by looking at P_{OPR} and P_{APR} together, it seemed that the best classifier was IRTS-EPF-G-g, which was confirmed by Fig. 11(b).

E. Houston Data

The Houston data are probably one of the most complicated datasets available on the website in terms of BKG and class classifiability. Fig. 12(a) and (b) shows the classification maps without BKG and with BKG produced by the 13 test methods,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

TABLE X

P_A, P_{OA}, P_{AA}, P_{PR}, P_{OPR}, AND P_{APR} PRODUCED BY IEPF-G-G, IRTS-EPF-G-G, PCAWSD, IRTS-3DCNN, IEPF-GABOR, IGPN(7), ILPN(6), AND IGLPN(7, 6) FOR U. OF PAVIA DATA (IGPN, ILPN, AND IGLPN WITH $n_{PC} = 15$ DETERMINED EMPIRICALLY AND $n_{PC} = 24$ DETERMINED BY VD)

	IEPF-G-g	IRTS-EPF-G-g	GFDN	PCA-WSD	IRTS-3DCNN	IEPF-Gabor	IGPN(7)	ILPN(6)	IGLPN (7,6)	IEPF-Gabor-VD	IGPN-VD(7)	ILPN-VD(6)	IGLPN-VD(7,6)
n_{PC}						15	15	15	15	24	24	24	24
Class	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$
1	94.1	97.79	95.84	95.19	99.07	99.01	99.37	98.24	99.31	98.79	98.85	97.79	**99.42**
2	92.13	98.41	93.06	99.33	97.24	98.01	**99.5**	97.51	99.33	98.06	99.61	98.58	99.17
3	96.05	95.73	99.32	99.59	**99.9**	97.89	99.52	97.17	99.72	96.46	99.65	98.7	99.81
4	**99.27**	98.51	96.58	98.18	97.71	94.29	93.99	93.64	93.06	95	93.91	92.91	91.76
5	98.66	99.01	99.8	**100**	99.95	99.98	99.8	99.93	99.88	99.98	99.95	99.85	**100**
6	94.54	99.07	96.57	99.88	98.32	99.58	**100**	**100**	**100**	99.65	**100**	**100**	**100**
7	98.39	99.82	99.87	100	99.92	**100**	**100**	99.92	99.97	**100**	**100**	99.97	**100**
8	93.96	97.82	94.97	**99.49**	99.34	95.4	98.8	98.43	99.11	94.71	98.83	97.49	98.98
9	97.75	98.1	99.79	**99.96**	99.93	98.66	97.64	98.52	96.06	98.03	97.18	98.27	97.36
P_{OA}	94.1	98.27	95.2	98.75	98.23	97.99	**99.07**	97.87	98.92	97.89	99.04	98.21	98.8
P_{AA}	96.09	98.25	97.31	**99.07**	99.04	98.09	98.74	98.15	98.49	97.85	98.66	98.17	98.5
P_{OPR}	19.41	20.27	19.63	20.37	20.26	20.21	**20.43**	20.19	20.4	20.19	**20.43**	20.26	20.38
P_{APR}	**33.18**	32.19	20.5	20.86	22.23	24.1	23.69	23.21	23.29	23.96	22.93	23.39	22.66
Time	102.59	82.53	20.9	9067.26	4759.33	26.04	64.02	92.38	96.7	26.73	58.08	54.08	82.39

IEPF-G-g, IRTS-EPF-G-g, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IGPN(3), ILPN(4), IGLPN(3, 2), IEPF-Gabor-VD, IGPN-VD(3), ILPN-VD(4), and IGLPN-VD(3, 2) with n_{PC}, determined empirically in [8, Supplementary Material] and determined by VD [49], respectively. Apparently, it is very difficult if not impossible to evaluate the classification performance of all 13 classifiers by visual inspection. In this case, we must rely on quantitative measures to conduct comparative analysis.

Table XI tabulates the results of a priori criteria, P_A, P_{OA}, and P_{AA}, and a posteriori criteria, APR and OPR, produced by the 13 test methods, IEPF-G-g, IRTS-EPF-G-g, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IGPN(7), ILPN(6), IGLPN(7, 6), IEPF-Gabor-VD, IGPN-VD(7), ILPN-VD(6), and IGLPN-VD(7, 6) with n_{PC} determined empirically and determined by VD, respectively, where $P_A(C_i)$ is the classification accuracy of class C_i. The experiments were run five times. Among all the 13 test methods, IGPN-VD(7) achieved the best results in terms of P_{OA}, and IGPN(7) achieved the best results in terms of P_{AA}. Although ILPN(4) did not show any significant improvement in performance compared to IEPF-Gabor. It is worth noting that the combination of GP and LP in IGLPN-VD(7, 6) led to enhanced accuracy for classes, 2, 9, 12, 13, and 15 compared to IEPF-Gabor-VD. More interestingly, IGPN(3) outperformed DL methods, GFDN, PCAWSD, and IRTS-3DCNN significantly in terms of accuracy.

In the comparison of the performance of n_{PC}, despite the fact that IGLPN using VD-determined n_{PC} performed slightly better than IGLPN using empirically determined n_{PC}, the use of VD offers an automatic rule for determining n_{PC}, eliminating the need for empirical determination through trial and error.

Interestingly, according to Table XI, when BKG was included for classification, P_{OPR} and P_{APR} were very low due to the complexity of its BKG. Specifically, all the P_{OPR} values were below 0.0236. As for P_{APR}, except IEPF-G-g with $P_{APR} = 22.17$ and IRTS-EPF-G-g with $P_{APR} = 27.74$, all other P_{APR} values were below 0.18.

Finally, if we compare the results in Table XI to results in Tables VIII–X for the other three datasets, a priori criteria, P_{OA} and P_{AA}, and a posteriori criteria, P_{OPR} and P_{APR}, for Houston data were much lower. This indicated that the Houston data are indeed a very challenging dataset for HSIC, particularly, classification with BKG included.

F. Robustness of IGLPN to Model Parameters

Upon implementing IGLPN, the model parameter, which is the number of used GLP layers, is the only parameter that needs to be selected empirically. To address this robustness issue, additional experiments were performed to evaluate classification accuracy across a range of pyramid layers [from (2, 1) to (7, 6)], as shown in Fig. 13, which reveals a clear dependence of classification accuracy on the number of pyramid layers. However, this dependence is determined by the size and class characteristics of the used dataset. For a large image such as Salinas, University of Pavia, and Houston datasets, we observed the classification accuracy increases with the number of pyramid layers and reaching their optimal classification accuracy when the layer number reaches (7, 6). This improvement is primarily due to additional layers in the GLP that provides a more detailed multiscale representation, enhancing edge detection and reducing redundant information, all of which significantly improve classification accuracy.

By contrast, for a dataset of small size such as the Purdue dataset, the optimal layer number was found to be (3, 2). The classification accuracy began to decrease afterward. This decline is attributed to the excessive downsampling caused by a higher number of layers, which results in an inevitable loss of essential image details and, thus, a negative impact on classification accuracy.

As a summary, the results in Fig. 13 suggested that an optimal number of pyramid layers is crucial and should be determined by the size of a dataset and its class characteristics. Otherwise, IGLPN showed robust performance across a variety of datasets when this parameter was appropriately adjusted.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

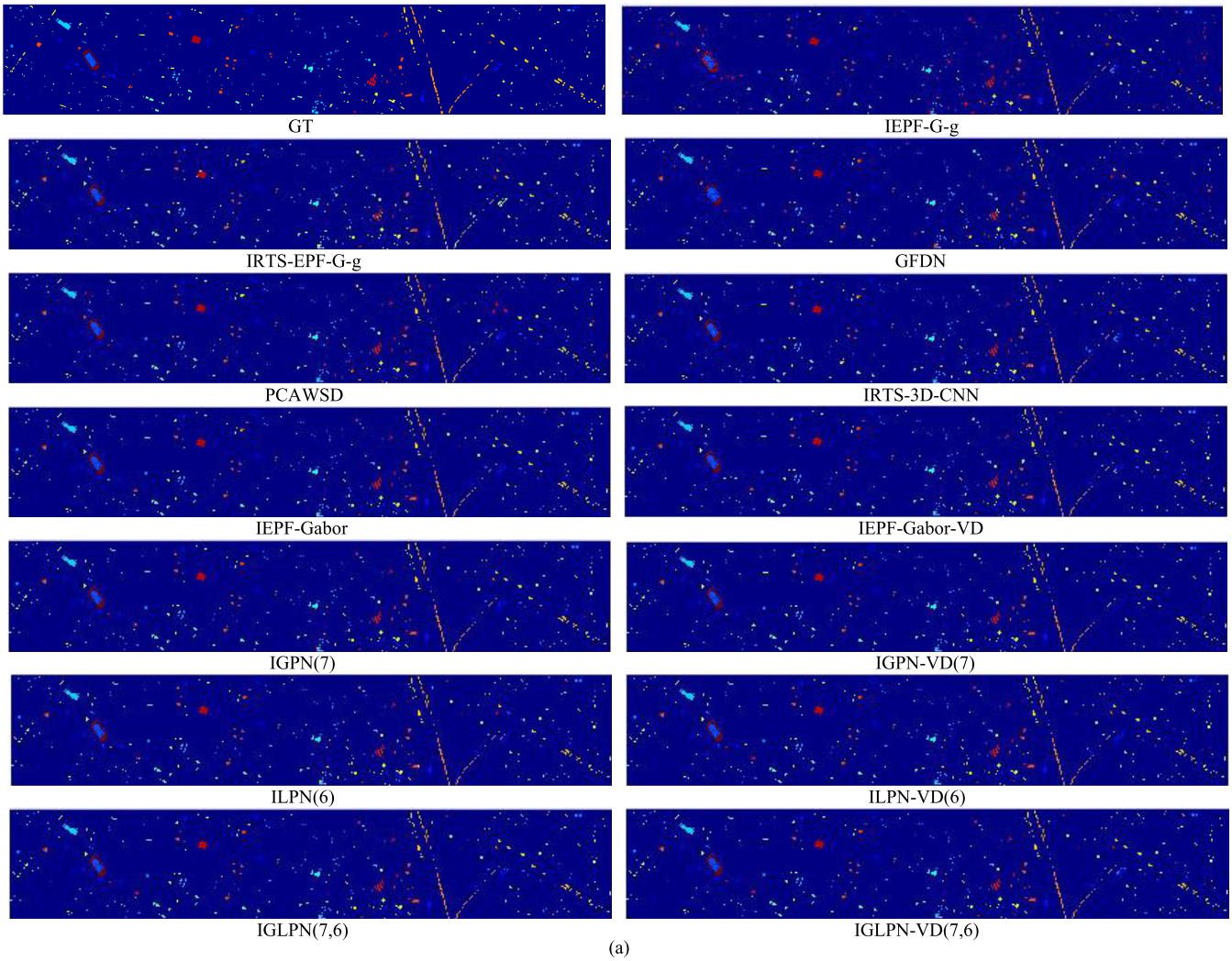


Fig. 12. Classification maps of by GT, IEPF-G-g, IRTS-EPF-G-g, GFDN, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IEPF-Gabor-VD, IGPN(7), IGPN-VD(7), ILPN(6), ILPN-VD(6), IGLPN(7, 6), and IGLPN-VD(7, 6) for the Houston data using 1% of training samples. (a) Without BKG. (b) With BKG.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

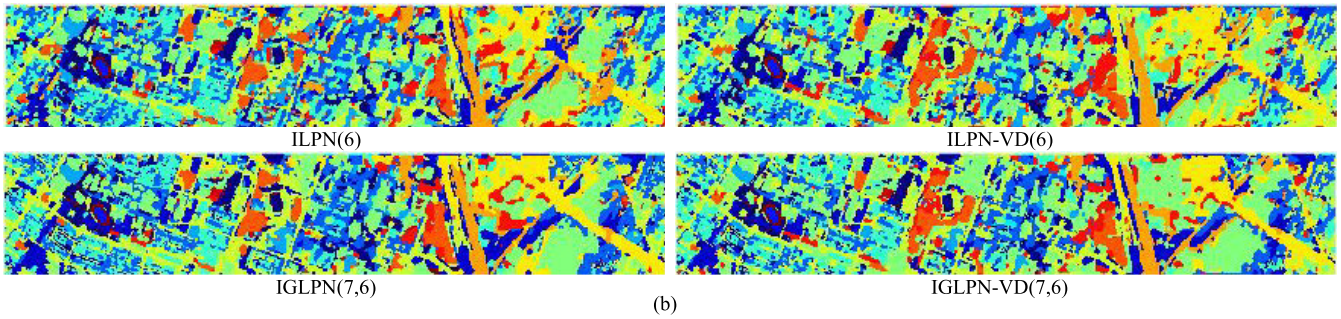


Fig. 12. *(Continued.)* Classification maps of by GT, IEPF-G-g, IRTS-EPF-G-g, GFDN, PCAWSD, IRTS-3DCNN, IEPF-Gabor, IEPF-Gabor-VD, IGPN(7), IGPN-VD(7), ILPN(6), ILPN-VD(6), IGLPN(7, 6), and IGLPN-VD(7, 6) for the Houston data using 1% of training samples. (a) Without BKG. (b) With BKG.

TABLE XI

P_A, P_{OA}, P_{AA}, P_{PR}, P_{OPR}, AND P_{APR} PRODUCED BY IEPF-G-G, IRTS-EPF-G-G, PCAWSD, IRTS-3DCNN, IEPF-GABOR, IGPN(7), ILPN(6), AND IGLPN(7, 6) FOR HOUSTON DATA (IGPN, ILPN, AND IGLPN WITH $n_{PC} = 15$ DETERMINED EMPIRICALLY AND $n_{PC} = 20$ DETERMINED BY VD)

	IEPF-G-g	IRTS-EPF-G-g	GFDN	PCA-WSD	IRTS-3DCNN	IEPF-Gabor	IGPN(7)	ILPN(6)	IGLPN (7,6)	IEPF-Gabor -VD	IGPN -VD(7)	ILPN -VD(6)	IGLPN -VD(7,6)
n_{PC}						15	15	15	15	20	20	20	20
Class	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$	$P_A(C_i)$
1	82.97	77.68	68.44	88.96	87.97	84.23	83.5	83.5	84.79	85.59	**91.36**	83.6	84.52
2	81.75	76.73	67.36	92.64	91.97	94.57	91.89	91.89	93.79	92.25	87.69	89.87	**94.61**
3	96.98	99.79	96.44	99.24	99.33	99.66	98.99	98.99	99.16	99.25	**99.87**	99.29	99.29
4	87.84	73.44	74.29	92.04	**97.92**	96.49	90.58	90.58	92.56	94.81	91.74	85.07	90.35
5	94.2	87.21	83.69	96.97	97.82	98.13	92.86	92.86	97.23	98.66	**100**	97.38	96.28
6	88.3	91.94	94.5	**94.4**	94	88.3	81.02	81.02	91.64	93.9	91.45	86.92	91.64
7	72.34	81.55	78.12	81.66	84.76	85.32	69.88	69.88	86.95	85.36	**89.54**	83.15	82.91
8	57.24	**96.51**	47.22	55.19	70.68	83.31	67.6	67.6	75.01	84.27	80.53	76.37	83.58
9	68.34	76.06	50.86	79.97	82.35	76.49	80.44	80.44	**88.24**	83.12	83.7	82.75	83.98
10	90	90.54	88.79	87.88	83.03	89.73	85.58	85.58	92.49	90.29	**98.62**	86.28	92.69
11	89.12	84.82	80.59	92.36	88.48	88.21	89.25	89.25	92.66	94.15	**94.51**	87.97	93.53
12	64.85	69.65	67.44	79.85	66.08	81.8	82.6	82.6	85.89	85.07	85.23	82.51	**88.08**
13	82.44	74.37	72.47	93.94	**96.94**	77.43	79.59	79.59	90.19	86.55	79.32	83.55	89.98
14	97.92	88.63	99.87	99.87	99.09	99.61	97.86	97.86	**100**	99.09	**100**	99.87	**100**
15	85.63	95.03	86.13	**99.75**	97.95	97.83	97.99	97.99	98.41	95.28	99.08	98.75	99.16
P_{OA}	80.76	83.01	73.9	87.06	87.23	88.61	90.89	84.98	90.29	90.26	**90.98**	87.09	90.36
P_{AA}	82.66	84.27	77.08	88.98	89.23	89.41	**92.12**	85.98	91.27	91.17	91.51	88.22	91.37
P_{OPR}	2.1	2.16	1.92	2.26	2.27	2.3	**2.36**	2.21	2.35	2.35	**2.36**	2.26	2.35
P_{APR}	22.19	**27.74**	3.59	6.25	11.93	14.18	9.72	7.78	9.26	17.68	12.95	11.45	14.8
Time	159.53	293.81	38.89	24798.54	13013.21	65.49	242.47	121.48	180.2	94.68	279.99	142.4	189.14

As for future work, it is highly desirable to develop an adaptive method to automatically determine the optimal number of pyramid layers according to the intrinsic characteristics of each dataset, so as to further improve the IGLPN's robustness and applicability.

G. Discussion

Finally, this section observed several interesting findings that collectively contribute to a comprehensive understanding of IGLPN.

1) First, the experimental section shows that IGLPN and IGPN produce very similar accuracies, which are significantly higher than those produced by ILPN. This suggests that using high-pass information provided by LP alone may introduce unwanted elements, such as amplified noise, thereby potentially reducing the classification accuracy. On the other hand, both IGLPN and IGPN consistently demonstrated robust classification outcomes across all four tested image scenes.
2) The experiments also demonstrated the advantages of using GP and LP in improving classification accuracy. The ability of these pyramidal structures to facilitate multiscale feature extraction is evident from the improvement in accuracy across different datasets. This reinforces their potential to capture intrinsic spatial details that play a pivotal role in accurate classification.
3) IGLPN diverges from traditional DL methods that often require a very large number of training samples to achieve high accuracy. On the contrary, IGLPN demonstrates a remarkable capability to attain high levels

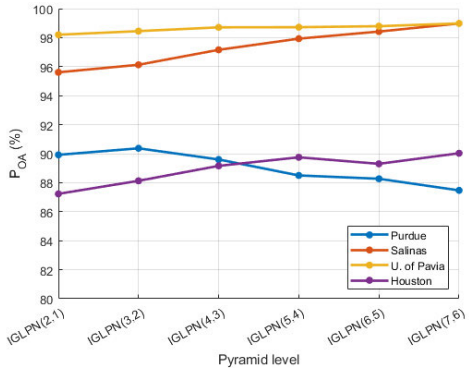


Fig. 13. Robustness of IGLPN to model parameters for four datasets.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

TABLE XII
P_{OA} OF THE ABLATION STUDY FOR FOUR HSIS

PCA	EPF	Gabor	GLP	IFP	IP	SA	PU	HU
×	√	√	√	√	65.67	96.27	85.81	
√	×	√	√	√	87.09	99.26	**99.51**	**92.27**
√	√	×	√	√	86.16	98.39	98.89	85.62
√	√	√	×	√	89.05	96.78	97.99	88.61
√	√	√	√	×	69.04	91.11	96.1	81.2
√	√	√	√	√	**90.17**	**99.32**	98.92	90.29

of accuracy without training samples to train the network. This ability shows the applicability of IGLPN to real-world scenarios where finding reliable training pools can be a constraint.

4) Furthermore, while IGLPN is inherently designed to mimic the structural attributes of CNN, a notable distinction lies in its computational efficiency. Unlike traditional DL methods, which often demand substantial computational time, IGLPN excels in this aspect. Its capacity to achieve compelling results while maintaining lower computational times shows its significant advantage of being practically feasible and resource-efficient. In conclusion, these findings show that IGLPN is a valuable technique in HSIC.
5) Compared to recent DL-based HSIC methods, IGLPN requires a smaller set of predetermined parameters, such as the parameters used for GFs tabulated in Table VII plus the TI threshold in (14). Compared to IGLPN, a typical DL model in similar applications often requires tuning a large set of parameters, including, but not limited to, filter size, patch size, batch size, stride, pooling size, the number of neurons, learning rate, epochs, and the number of convolution blocks.
6) Moreover, IGLPN does not use weights to train its network as CNN uses backward propagation to adjust a large set of learnable parameters (e.g., the weight of the convolutional kernel). Instead, IGLPN replaces backpropagation with an IFP, which leads to better convergence speed and reduces model complexity in comparison with a standard CNN model.

VIII. ABLATION STUDIES OF SUBCOMPONENTS IN IGLPN

In order to evaluate the significance of each subcomponent within IGLPN, we conducted an ablation study in this section to assess the contributions of five subcomponents: PCA, EPF, Gabor filtering, GLP, and IFP implemented in IGLPN.

The parameters for IGLPN in these ablation studies are the same as tabulated in Table VII. It is worth noting that when EPF is excluded from IGLPN, the standard SVM is employed as a substitute to produce binary classification maps. Table XII tabulates $P_{OA}(\%)$ for five subcomponents used in IGLPN based on four datasets where a "√" symbol and a "×" symbol under a subcomponent indicate that the subcomponent is and is not implemented, respectively, where IP, SA, PU, and HU are the abbreviations of Purdue data, Salinas, U. of Pavia, and Houston. It should be noted that there was no result when PCA was applied to Houston data because the removal of PCA from IGLPN results in an out-of-memory error.

Overall, Table XII provides hard evidence that all the subcomponents played their own important roles in retaining the classification performance of IGLPN. In particular, IFP is the most important subcomponent that leads to the greatest improvement on IGLPN due to updating classification information of the processed data cube by incorporating GFSMaps in each iteration. Another interesting finding is that replacing EPF with standard SVM results in a degradation in the P_{OA} for IP. However, this replacement does not significantly affect the classification performance of the other three datasets (SA, PU, and HU). This finding indicates that the classifier used in the first stage of IGLPN is indeed flexible and allows various spectral classifiers or spectral–spatial classifiers in place of EPF.

IX. CONCLUSION

GP is a commonly used image coding technique in image compression and image progressive transmission in signal processing/communications. Its idea is to use Gaussian windows as low-pass filters to construct a GP, which reduces spatial resolution and image size layer by layer. In the meantime, GP also constructs a corresponding LP, which retains innovational information between two consecutive layers in GP for image reconstruction. CNN is a widely used machine learning technique in pattern classification. Its idea is to use a CL to capture spatial information of a receptive field and a PL to subsample the CL-processed data. Interestingly, these two operations, CL and PL, can be realized by Gaussian-window-based low-pass filters to reduce spatial resolution and subsampling to reduce image size implemented in GP, respectively. By virtue of this interpretation, (CL, PL) of a single layer in CNN can be realized by a single layer in GP. In particular, GP implements CL and PL altogether simultaneously instead of CL and PL that are implemented one after another separately as two sublayers in a single layer in CNN. This article investigates the relationship and similarities between these two seemingly different techniques in applications of HSIC and further presents an iterative GLP-based classification network, called IGLPN, which is a new paradigm as an alternative to ICNN. Specifically, IGLPN iteratively processes two pieces of information. One is predicted information produced by GP in correspondence to CNN. The other is LP, which is absent in CNN. Such an LP provides innovational information to improve classification performance. As a result, IGLPN has advantages over ICNN in the sense that it not only can use GP, LP, or both to enhance its classification performance but also can capture spatial correlation among classified data samples via feedback loops iteratively. Also, IGLPN using the fixed weights by GP has an advantage over adaptive weights trained by training samples in CNN. This is because adaptive weights are determined by training samples and parameters used in CNN, which must be fine-tuned. However, how to select parameters and judiciously select an appropriate set of training samples is challenging for CNN.

In addition, CNN does not use feedback information to update data cubes as IGLPN does. As a final remark, despite the fact that both ICNN and IGLPN implement a feedback

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

network to capture spatial information of classified data samples, fundamentally speaking, they are quite different networks. IGLPN includes GFs to extract eight-orientation information that can be realized by the human visual system, which does not exist in ICNN. Moreover, since ICNN requires training samples to train weights connecting between layers, it has an explainability issue that IGLPN does have.

REFERENCES

[1] S. Li, W. Song, L. Fang, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Deep learning for hyperspectral image classification: An overview," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6690–6709, Sep. 2019.

[2] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Deep learning classifiers for hyperspectral imaging: A review," *ISPRS J. Photogramm. Remote Sens.*, vol. 158, pp. 279–317, Dec. 2019.

[3] W. Hu, Y. Huang, W. Li, F. Zhang, and F. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, Jul. 2015, Art. no. 258619.

[4] P. Burt and E. Adelson, "The Laplacian pyramid as a compact image code," *IEEE Trans. Commun.*, vol. COM-31, no. 4, pp. 532–540, Apr. 1983.

[5] C.-I. Chang, C.-C. Liang, and P. F. Hu, "Iterative random training sampling convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5513526.

[6] X. Kang, S. Li, and J. A. Benediktsson, "Spectral–spatial hyperspectral image classification with edge-preserving filtering," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 5, pp. 2666–2677, May 2014.

[7] C.-I. Chang, *Real-Time Recursive Hyperspectral Sample and Band Processing: Algorithm Architecture and Implementation*. New York, NY, USA: Springer, 2017, ch. 3.

[8] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[9] Q. Zhu et al., "A spectral–spatial-dependent global learning framework for insufficient and imbalanced hyperspectral image classification," *IEEE Trans. Cybern.*, vol. 52, no. 11, pp. 11709–11723, Nov. 2022.

[10] S. K. Roy, S. Manna, T. Song, and L. Bruzzone, "Attention-based adaptive spectral–spatial kernel ResNet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7831–7843, Sep. 2021.

[11] C. Yu, R. Han, M. Song, C. Liu, and C.-I. Chang, "Feedback attention-based dense CNN for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 550191.

[12] C. Yu, R. Han, M. Song, C. Liu, and C.-I. Chang, "A simplified 2D–3D CNN architecture for hyperspectral image classification based on spatial–spectral fusion," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 2485–2501, 2020.

[13] H. Yu, H. Zhang, Y. Liu, K. Zheng, Z. Xu, and C. Xiao, "Dual-channel convolution network with image-based global learning framework for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[14] M. E. Paoletti, J. M. Haut, X. Tao, J. Plaza, and A. Plaza, "FLOP-reduction through memory allocations within CNN for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5938–5952, Jul. 2021.

[15] H. Yu, Z. Xu, K. Zheng, D. Hong, H. Yang, and M. Song, "MSTNet: A multilevel spectral–spatial transformer network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5532513.

[16] U. A. Bhatti et al., "Local similarity-based spatial–spectral fusion hyperspectral image classification with deep CNN and Gabor filtering," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 514215.

[17] Y. Jiang, Y. Li, S. Zou, H. Zhang, and Y. Bai, "Hyperspectral image classification with spatial consistence using fully convolutional spatial propagation network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 12, pp. 10425–10437, Dec. 2021.

[18] X. Kang, C. Li, S. Li, and H. Lin, "Classification of hyperspectral images by Gabor filtering based deep network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 4, pp. 1166–1178, Apr. 2018.

[19] C. Liu, J. Li, L. He, A. J. Plaza, S. Li, and B. Li, "Naive Gabor networks for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 376–390, Jan. 2021.

[20] X. Zhao, R. Tao, W. Li, W. Philips, and W. Liao, "Fractional Gabor convolutional network for multisource remote sensing data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5503818.

[21] E. K. Ghasrodashti, M. S. Helfroush, and H. Danyali, "Sparse-based classification of hyperspectral images using extended hidden Markov random fields," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 11, pp. 4101–4112, Nov. 2018.

[22] M. E. Paoletti, J. M. Haut, R. Fernandez-Beltran, J. Plaza, A. J. Plaza, and F. Pla, "Deep pyramidal residual networks for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 740–754, Feb. 2019.

[23] B. Tu, X. Liao, Q. Li, C. Zhou, and A. Plaza, "Multi-resolution pyramid enhanced non-local feature extraction for hyperspectral classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 15, pp. 5865–5879, 2022.

[24] Z. He, Q. Shi, K. Liu, J. Cao, W. Zhan, and B. Cao, "Object-oriented mangrove species classification using hyperspectral data and 3-D Siamese residual network," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 12, pp. 2150–2154, Dec. 2020.

[25] H. Shi, G. Cao, Z. Ge, Y. Zhang, and P. Fu, "Double-branch network with pyramidal convolution and iterative attention for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 7, p. 1403, Apr. 2021.

[26] J. Yue, D. Zhu, L. Fang, P. Ghamisi, and Y. Wang, "Adaptive spatial pyramid constraint for hyperspectral image classification with limited training samples," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5512914.

[27] C. Shi, D. Liao, Y. Xiong, T. Zhang, and L. Wang, "Hyperspectral image classification based on dual-branch spectral multiscale attention network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 10450–10467, 2021.

[28] R. Shang et al., "Hyperspectral image classification based on pyramid coordinate attention and weighted self-distillation," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5544316.

[29] D. Song, C. Yang, B. Wang, J. Zhang, H. Gao, and Y. Tang, "SSRNet: A lightweight successive spatial rectified network with noncentral positional sampling strategy for hyperspectral images classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5519115, doi: 10.1109/TGRS.2023.3301310.

[30] Y. Tang, X. Xie, and Y. Yu, "Hyperspectral classification of two-branch joint networks based on Gaussian pyramid multiscale and wavelet transform," *IEEE Access*, vol. 10, pp. 56876–56887, 2022, doi: 10.1109/ACCESS.2022.3172501.

[31] S. Li, Q. Hao, X. Kang, and J. A. Benediktsson, "Gaussian pyramid based multiscale feature fusion for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 9, pp. 3312–3324, Sep. 2018.

[32] Q. Hao, B. Sun, S. Li, M. M. Crawford, and X. Kang, "Curvature filters-based multiscale feature extraction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5507916.

[33] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2018.

[34] S. Zhong, C.-I. Chang, and Y. Zhang, "Iterative edge preserving filtering approach to hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 1, pp. 90–94, Jan. 2019.

[35] P. Duan, X. Kang, S. Li, P. Ghamisi, and J. A. Benediktsson, "Fusion of multiple edge-preserving operations for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 12, pp. 10336–10349, Dec. 2019.

[36] X. Kang, X. Xiang, S. Li, and J. A. Benediktsson, "PCA-based edge-preserving features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 12, pp. 7140–7151, Dec. 2017.

[37] K. He, J. Sun, and X. Tang, "Guided image filtering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 6, pp. 1397–1409, Jun. 2013.

[38] J. G. Daugman, "Complete discrete 2-D Gabor transforms by neural networks for image analysis and compression," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 36, no. 7, pp. 1169–1179, Jun. 1988.

[39] A. K. Jain and F. Farrokhnia, "Unsupervised texture segmentation using Gabor filters," *Pattern Recognit.*, vol. 24, no. 12, pp. 1167–1186, 1991.

[40] P. Kruizinga and N. Petkov, "Nonlinear operator for oriented texture," *IEEE Trans. Image Process.*, vol. 8, no. 10, pp. 1395–1407, Oct. 1999.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.

[41] S. Theodoridis and K. Koutroumbas, *Pattern Recognition*. New York, NY, USA: Academic, 1999, p. 366.
[42] C.-I. Chang, "A statistical detection theory approach to hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2057–2074, Apr. 2019.
[43] C.-I. Chang, K. Y. Ma, C.-C. Liang, Y.-M. Kuo, S. Chen, and S. Zhong, "Iterative random training sampling spectral spatial classification for hyperspectral images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 3986–4007, 2020.
[44] C.-I. Chang, *Hyperspectral Imaging: Techniques for Spectral Detection and Classification*. New York, NY, USA: Kluwer Academic/Plenum Publishers, 2003.
[45] C.-I. Chang and Q. Du, "Estimation of number of spectrally distinct signal sources in hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 3, pp. 608–619, Mar. 2004.
[46] C.-I. Chang, "A review of virtual dimensionality for hyperspectral imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 4, pp. 1285–1305, Apr. 2018.
[47] S. Zhong et al., "Class feature weighted hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 12, pp. 4728–4745, Dec. 2019.
[48] (Sep. 2023). [Online]. Available: http://xudongkang.weebly.com/
[49] C. Yu, Y. Wang, M. Song, and C.-I. Chang, "Class signature-constrained background-suppressed approach to band selection for classification of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 1, pp. 14–31, Jan. 2019.



Chein-I Chang (Life Fellow, IEEE) received the Ph.D. degree in electrical engineering from the University of Maryland, College Park, MD, USA, in 1987.

Dr. Chang is currently a Professor with the University of Maryland, Baltimore County (UMBC), Baltimore, MD, USA. He has been a Chang Jiang Scholar Chair Professor and the Director of the Center for Hyperspectral Imaging in Remote Sensing (CHIRS), Dalian Maritime University, Dalian, China, since 2016, and a Yushan Scholar Chair Professor with National Cheng Kung University, Tainan, Taiwan. He has been a Chair Professor with National Yang Ming Chiao Tung University, Hsinchu, Taiwan, since 2019. He has authored four books, *Hyperspectral Imaging: Techniques for Spectral Detection and Classification* (Kluwer Academic Publishers, 2003), *Hyperspectral Data Processing: Algorithm Design and Analysis* (John Wiley & Sons, 2013), *Real-Time Progressive Hyperspectral Image Processing: Endmember Finding and Anomaly Detection* (Springer, 2016), and *Recursive Hyperspectral Sample and Band Processing: Algorithm Architecture and Implementation* (Springer, 2017), and a forthcoming book, *Statistical Hyperspectral Target and Anomaly Detection: Signal and Image Processing Perspectives* (Wiley, 2025). He also edited three books: *Recent Advances in Hyperspectral Signal and Image Processing* (2006), *Hyperspectral Data Exploitation: Theory and Applications* (John Wiley & Sons, 2007), and *Advances in Hyperspectral Image Processing Techniques* (Wiley, 2023). He also co-edited with A. Plaza the book *High Performance Computing in Remote Sensing* (CRC Press, 2007). His research interests include multispectral/hyperspectral image processing, automatic target recognition, and medical imaging.

Dr. Chang is a fellow of Society for Photo-optical Instrumentation Engineers (SPIE). He is also serving as an Associate Editor for *Remote Sensing* and IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING.



Chia-Chen Liang received the B.S. degree in computer science from the National University of Kaohsiung, Kaohsiung, Taiwan, in 2014, the M.S. degree in electrical engineering from National Chung Hsing University, Taichung, Taiwan, in 2017, and the M.S. degree in electrical engineering from the University of Maryland, Baltimore County, Baltimore, MD, USA, in 2023, where she is currently pursuing the Ph.D. degree in electrical engineering with the Remote Sensing Signal and Image Processing Laboratory.

Her research interests include hyperspectral image classification, pattern recognition, and deep learning.



Peter Fuming Hu received the B.S. degree in electrical engineering from Shanghai University, Shanghai China, in 1984, and the M.S. and Ph.D. degrees in computer science from the University of Maryland, Baltimore County (UMBC), Baltimore, MD, USA, in 1992 and 2013, respectively.

He has more than 30 years of experience in clinical research at the R Adams Cowley Shock Trauma Center, University of Maryland School of Medicine, Baltimore. During that time, he worked as a principal investigator (PI) or a co-PI on over 50 successfully completed peer-reviewed grants funded by various U.S. government agencies. He has been with the University of Maryland School of Medicine since 1992, where he is currently a Professor with the Department of Anesthesiology, Surgery and Epidemiology. He is also an Adjunct Professor with UMBC. He is also the Chief Technologist with the University of Maryland R Adams Cowley Shock Trauma Center, University of Maryland Medical Center. He has published over 150 peer-reviewed papers. His research interests include applying big data machine learning methods for the prediction of trauma patient outcomes.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 21:28:42 UTC from IEEE Xplore. Restrictions apply.